

Министерство образования и науки Российской Федерации

Федеральное государственное бюджетное образовательное учреждение
высшего профессионального образования

ТОМСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ СИСТЕМ
УПРАВЛЕНИЯ И РАДИОЭЛЕКТРОНИКИ (ТУСУР)

Кафедра комплексной информационной безопасности
электронно-вычислительных систем (КИБЭВС)

Н.А. Новгородова

**СОЗДАНИЕ ПРИЛОЖЕНИЯ В MICROSOFT VISUAL STUDIO
ДЛЯ РАБОТЫ СО СВЯЗАННЫМИ ТАБЛИЦАМИ БАЗЫ ДАННЫХ**

**Учебно-методические указания к выполнению лабораторных работ
по дисциплинам
«Безопасность систем баз данных»,
«Базы данных и экспертные системы»,
«Базы данных»**

Томск 2015

ЛАБОРАТОРНАЯ РАБОТА №3

СОЗДАНИЕ ПРИЛОЖЕНИЯ В MICROSOFT VISUAL STUDIO ДЛЯ РАБОТЫ СО СВЯЗАННЫМИ ТАБЛИЦАМИ БАЗЫ ДАННЫХ

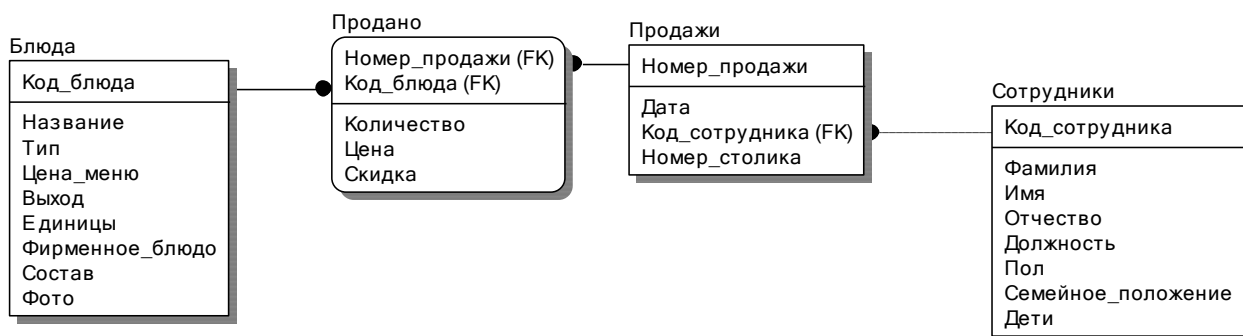
Цель работы:

Изучение основных особенностей создания приложения для работы со связанными таблицами базы данных в Microsoft Visual Studio для своей предметной области (на примере базы данных для кафе).

1 Введение

Разрабатываемое приложение предназначено для работы с базой данных для кафе (см. лабораторную работу №1, лабораторную работу №2).

Структура БД для кафе в рассматриваемом примере:



В лабораторной работе №2 было создано приложение для работы с данными в родительских таблицах:

- **Блюда;**
- **Сотрудники.**

В данных таблицах нет внешних ключей (FK), они являются родительскими таблицами:

- таблица **Блюда** связана с таблицей **Продано** связью 1:M;
- таблица **Сотрудники** связана с таблицей **Продажи** связью 1:M;
- таблица **Продажи** связана с таблицей **Продано** связью 1:M.

Примечание:

Таблица **Продано** появилась в БД из-за существования связи M:M между **Блюда** и **Продажи**.

В БД для кафе была установлена **ссылочная целостность**, для которой характерно:

- первичный ключ (PK) таблицы должен содержать уникальные

непустые значения для данной таблицы;

- внешний ключ (FK) потомка должен содержать только те значения, которые уже имеются среди значений PRIMARY KEY родителя (или колонок UNIQUE родителя);

- нельзя удалить экземпляр родителя, имеющий хотя бы один связанный с ним экземпляр потомка. Чтобы удалить экземпляр родителя, имеющего связанные потомки, необходимо сначала удалить связанные записи в потомке, а потом удалить экземпляр родителя с ними связанного. ИЛИ Можно установить механизм **каскадного удаления**, при котором удаляется экземпляр родителя вместе со связанными экземплярами потомка;

- нельзя изменить значение в PK (UNIQUE) родителя, имеющем хотя бы один связанный с ним экземпляр потомка. Можно установить механизм **каскадного обновления**, при котором изменяется PK (UNIQUE) родителя и значения FK связанных экземпляров потомка.

Ограничение FOREIGN KEY может содержать пустые значения (значения NULL). Кроме того, тип данных и размер колонки или колонок внешнего ключа должны совпадать со ссылочной колонкой или колонками.

Для связи между таблицами **Сотрудники** и **Продажи** механизм **каскадного обновления связанных записей** не установлен, так как Код_сотрудника в **Сотрудники** является типом данных счетчик, а его нельзя редактировать.

Для связи между таблицами **Сотрудники** и **Продажи** механизм **каскадного удаления связанных записей** не установлен, чтобы при удалении данных о сотруднике, связанные с ним продажи автоматически не удалялись. Чтобы удалить сотрудника необходимо сначала удалить все его продажи, а затем сведения по самому сотруднику.

Для связи между таблицами **Продажи** и **Продано** механизм **каскадного обновления связанных записей** не установлен, так как Номер_продажи в **Продажи** является типом данных счетчик, а его нельзя редактировать.

Для связи между таблицами **Продажи** и **Продано** установлен механизм **каскадного удаления связанных записей**, чтобы при удалении данных о продаже, связанные с ней блюда в **Продано** автоматически удалялись.

Для связи между таблицами **Блюда** и **Продано** установлен механизм **каскадного обновления связанных записей**, так как Код_блюда в **Блюда** может меняться (например, введение новой формы меню, а блюдо неизменно). При этом необходимо, чтобы связанные с данным блюдом продажи автоматически среагировали на смену значения в Код_блюда.

Механизм **каскадного удаления связанных записей** для связи между таблицами **Блюда** и **Продано** не установлен, чтобы при удалении данных о блюде, связанные с ним продажи в **Продано**, автоматически не удалялись.

2 Практические задания

Работа продолжается в проекте, созданном при выполнении лабораторной работы №2.

2.1 Связь 1:М между Продажи и Продано в приложении

Как было сказано ранее, в БД кафе таблица **Продажи** связана с таблицей **Продано** связью 1:М.

Покажем, как можно, например, реализовать данную связь в приложении.

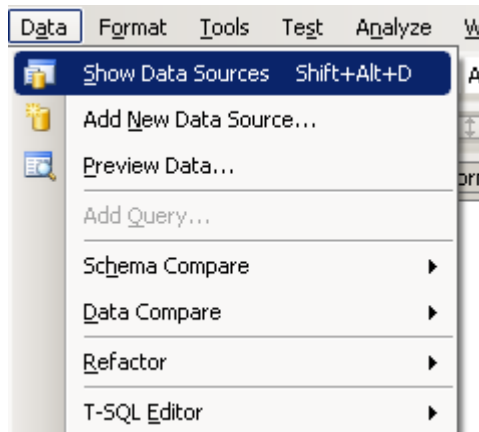
Создайте для работы с данными продаж отдельную форму.

Для этого нажмите на проекте правую кнопку мыши и выберите пункт **Add\Windows Forms**.

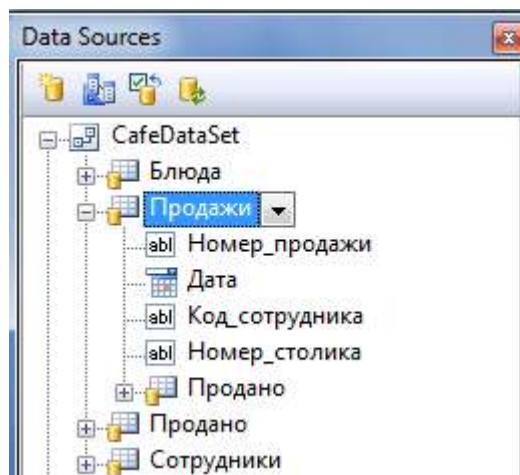
Задайте для новой формы имя **FormSalesList**.

Свойство **Text** для новой формы задайте в **Продажи**.

Откройте источник данных **DataSource**, выбрав меню **Data\Show Data Source**:

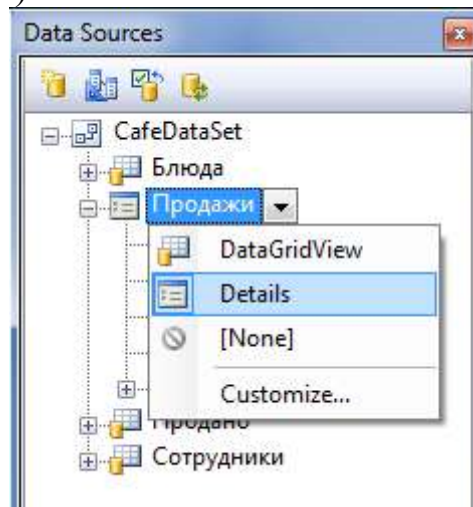


В открытом источнике данных выберите таблицу **Продажи**:



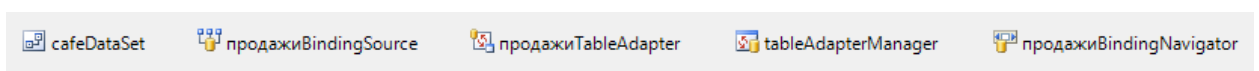
Обратите внимание, что для таблицы **Продажи** при нажатии + открывается не только список колонок данной таблицы, но и связанная с ней таблица **Продано**.

Выберете для продаж форму представления данных **Details** (каждое поле в отдельном компоненте):



И удерживая левую кнопку мыши, положите на форму. В результате получится форма:

И создадутся объекты:

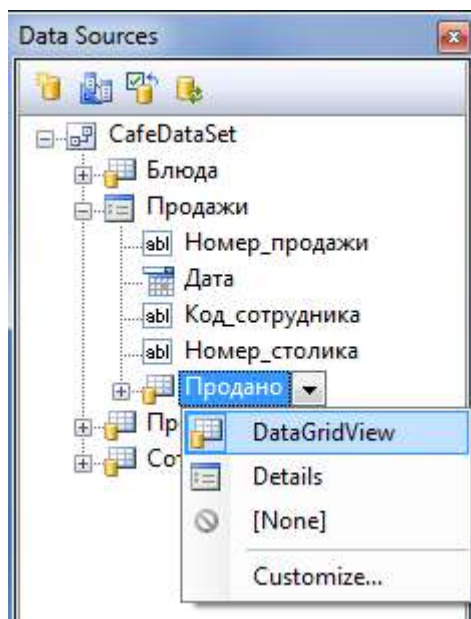


Разместите с панели инструментов **Toolbox** компонент **groupBox** для **Продано**.

Для **groupBox** укажите:

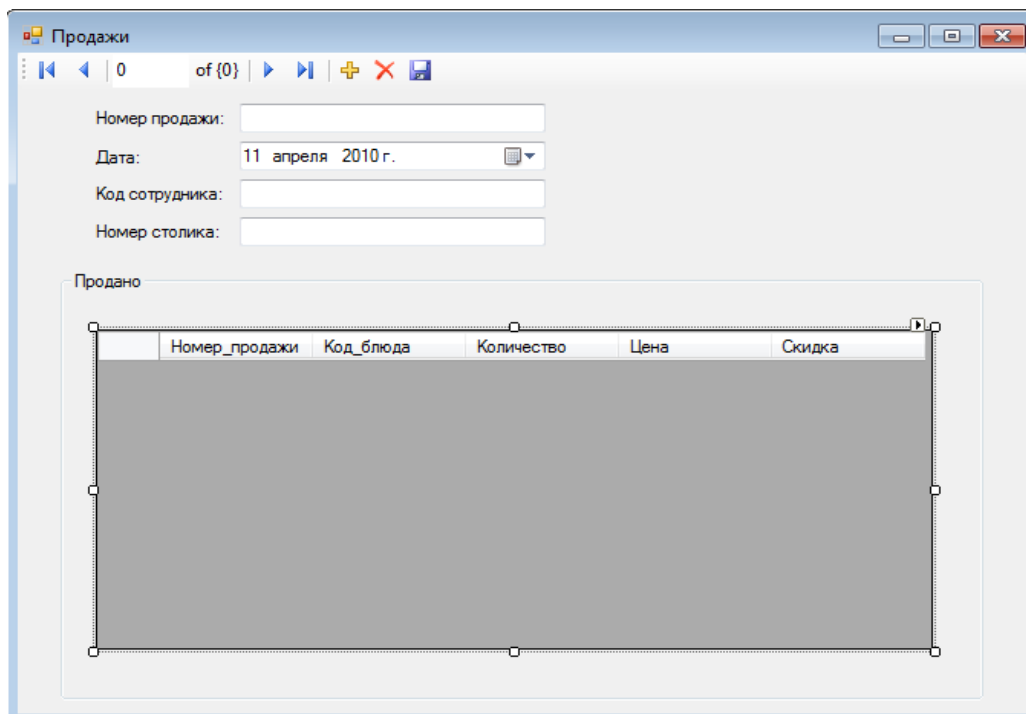
- свойство **Name** в **groupBoxDishSale**;
- свойство **Text** в **Продано**.

Выберете для таблицы **Продано** в **Продажи** (чтобы показывались данные по проданным блюдам для выбранной пользователем продажи) форму представления данных в **DataGridView**:



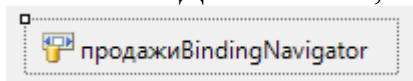
И перенесите, удерживая левую кнопку мыши, на **groupBoxDishSale**.

В результате получится форма:



Обратите внимание, что **BindingNavigator** для **Продано** не был размещен. Добавим его сами. Для этого, например, выделите

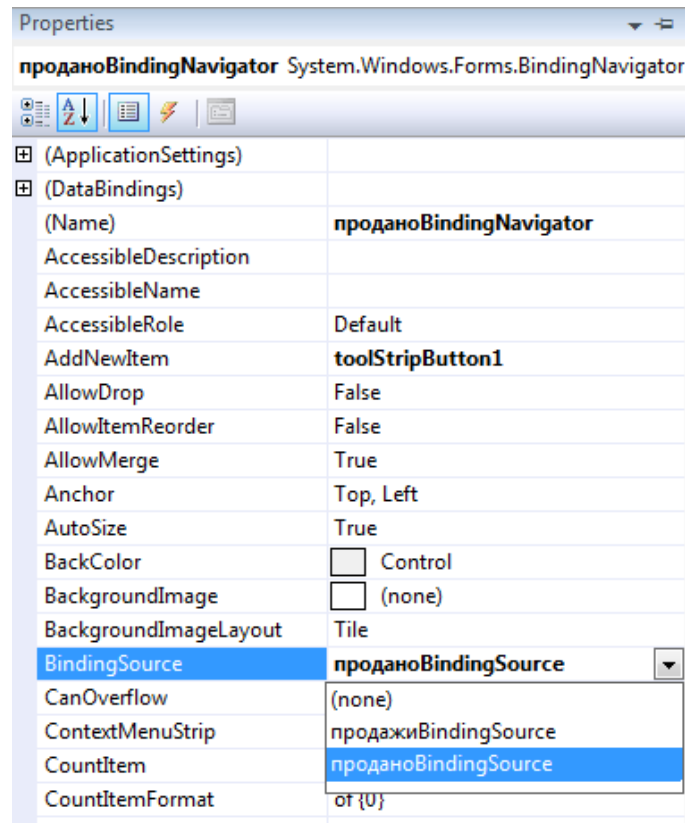
продажиBindingNavigator



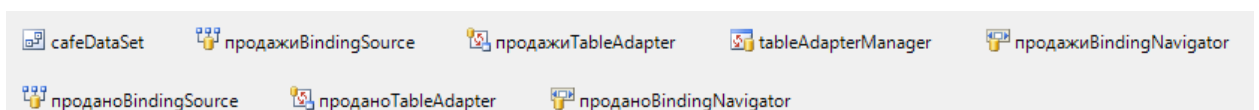
и нажмите на нем

правую кнопку мыши и выберете команду копировать (**Copy**). Затем на **groupBoxDishSale** нажмите правую кнопку мыши и выберете команду вставить (**Paste**). В результате будет добавлен **bindingNavigator1**. Для **bindingNavigator1** укажите:

- свойство **Name** в **проданоBindingNavigator**;
- в свойстве **BindingSource** укажите **проданоBindingSource** для привязки к **Продано**:



В итоге получаем объекты:



Настройте размещение **проданоBindingNavigator** и **проданоDataGridView** в **groupBoxDishSale** через свойство **Doc**:

- для **проданоBindingNavigator** свойство **Doc** в **Top** (верх);
- для **проданоSaleDataGridView** свойство **Doc** в **Fill** (вся область).

Настройте более удобный вид для пользователей **проданоBindingNavigator** и **продажиBindingNavigator** (см. лабораторную работу №2).

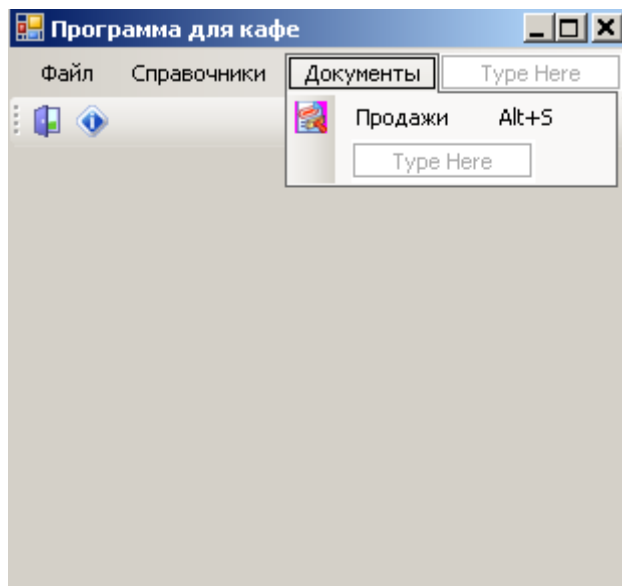
Задайте для кнопки сохранения данных в **проданоBindingNavigator** свойство **Name** в **проданоBindingNavigatorSaveItem**.

Запрограммируйте кнопку для сохранения данных для проданоBindingNavigatorSaveItem (по аналогии с продажиBindingNavigatorSaveItem):

```
private void проданоBindingNavigatorSaveItem_Click(object sender, EventArgs e)
{
    this.Validate();
    this.проданоBindingSource.EndEdit();
    this.tableAdapterManager.UpdateAll(this.cafeDataSet);
}
```

Покажем форму для продаж из нашей главной формы.

Для этого перейдите на главную форму и оформите для Продаж специальные пункты меню и панели инструментов (см. лабораторную работу №2). Например, в главном меню:



Реализуем шаблон проектирования «одинокка» » (англ. **Singleton**) (см. лабораторную работу №2) для данной формы.

Для этого в коде формы FormSalesList пропишите:

```
static FormSalesList f;
static public FormSalesList fs
{
    get
    {
        if (f == null || f.IsDisposed) f = new FormSalesList();
        return f;
    }
}

public void ShowForm()
{
}
```



```

        Show();
        Activate();
    }

```

Тогда на главной форме для пункта меню Продажи пропишите код:

```

private void продажиToolStripMenuItem_Click(object sender, EventArgs e)
{
    FormSalesList.fs.ShowForm();
}

```

Запустите программу и посмотрите появившуюся возможность.

Примечание:

посмотрите, что возможен переход между формами. В случае повторного выбора пункта меню Продажи будет показана и сфокусирована одна форма для Продаж.

Запустите программу и посмотрите, как отображается данная форма, как работать с данными в данной форме. Обратите внимание, что показываются проданные блюда именно выбранной продажи:

Номер_продажи	Код_блюда	Количество	Цена	Скидка
1	1	10	120,0000	0,20
1	2	4	140,0000	0,00
1	3	5	120,0000	0,10
1	7	2	130,0000	0,00
*				

Примечание:

Чтобы при добавлении новой продажи, дата продажи (`cafeDataSet.Продажи.Columns["Дата"]`) была присвоена по умолчанию (`DefaultValue`) текущей дате (`DateTime.Now`) допишите в обработчике

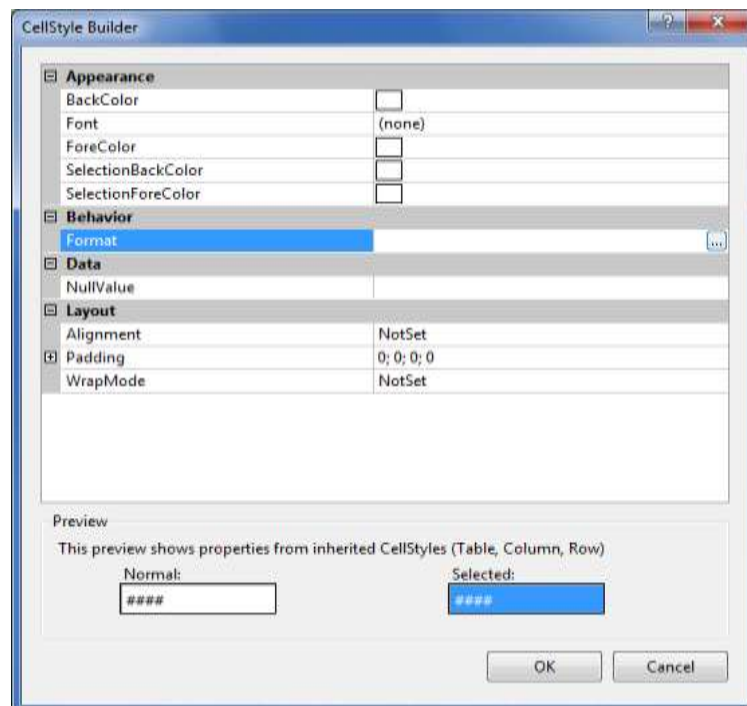
```
private void FormSalesList_Load(object sender, EventArgs e)
```

события на загрузку формы **FormSalesList** следующий код:

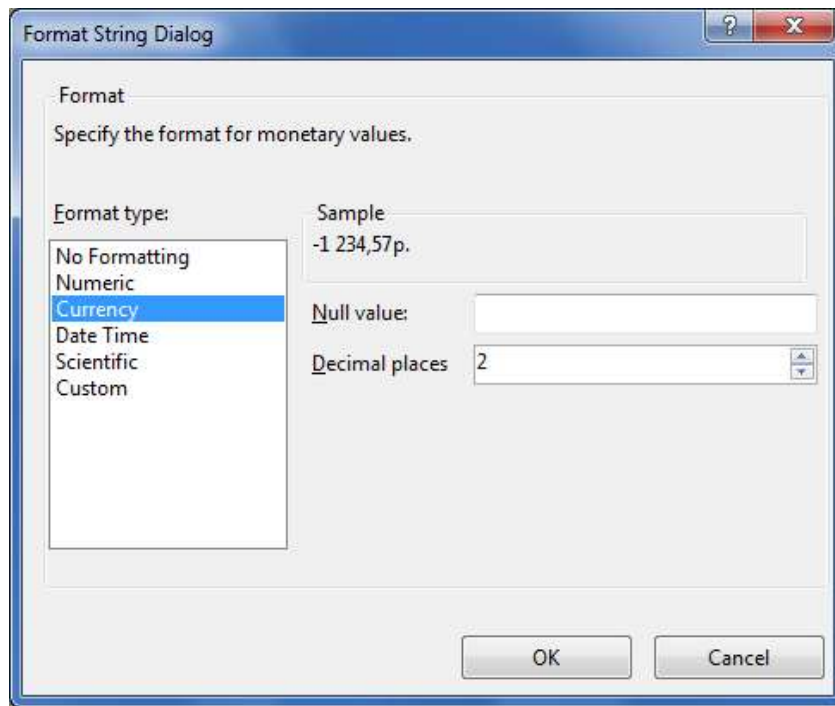
```
cafeDataSet.Продажи.Columns["Дата"].DefaultValue = DateTime.Now;
```

Для данной БД **Цена** должна быть выражена в денежном формате с точностью в 2 знака.

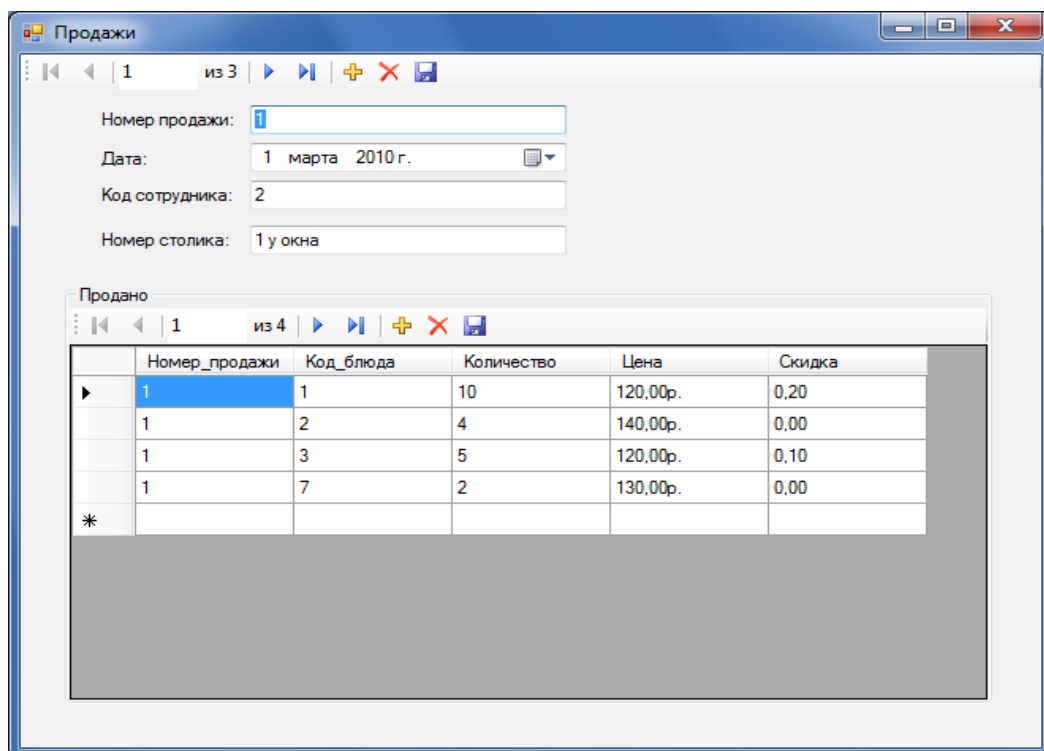
Чтобы это настроить зайдите в редактор колонок **проданоDataGridView**. Для колонки **Цена** зайдите в свойство **DefaultCellStyle**. В котором определите через свойство **Format**:



тип денежный (**Currency**) с точностью в 2 знака:



Запустите программу. Посмотрите настроенную возможность.



2.2 Обработка исключений в приложении

Ошибки, вызываемые компьютерной программой, могут быть разделены на три основных типа:

- **Design-time** (во время проектирования) - возникают при написании кода, который не удовлетворяет правилам того языка, на котором он пишется.

- **Runtime** (во время выполнения) - возникают при работе с программой, когда проводится попытка выполнить недопустимое действие, например, нарушение целостности данных, доступ к данным, которых не существуют, отсутствие прав доступа и др.

- **Logic** (логические) - не оказывают никакого влияния на работоспособность приложения. Наличие в программе логических ошибок означает лишь то, что могут быть неверные результаты от работы программы. Например, ошибки в вычислениях и др.

Чаще всего при работе с программой пользователи совершают **Runtime** ошибки.

Runtime ошибки вызывают **исключения (exception)**.

Исключение (exception) – это реакция на ошибку, сгенерированную приложением.

При возникновении исключения появляется диалог, сообщающий об ошибке с дополнительной информацией. Этот диалог дает возможность пользователю продолжить выполнение программы.

Обработчик ошибок – это блок кода, который может обнаружить исключения и выполнить необходимые действия для исправления этой ошибки.

Чтобы их обработать, можно использовать различные конструкции, например, конструкцию **try...catch**

Если в блоке **try** генерируется исключение, то программа автоматически переходит в блок **catch**. Если никаких исключений не генерируется, то программа пропускает блок **catch**.

Покажем обработку исключений на примере работы с формой Продажи:

Продажи

1 из 3

Номер продажи: 1

Дата: 1 марта 2010 г.

Код сотрудника: 2

Номер столика: 1 у окна

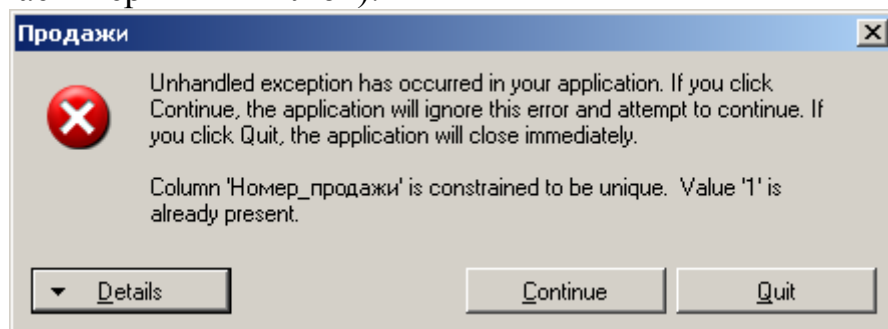
Продано

1 из 4

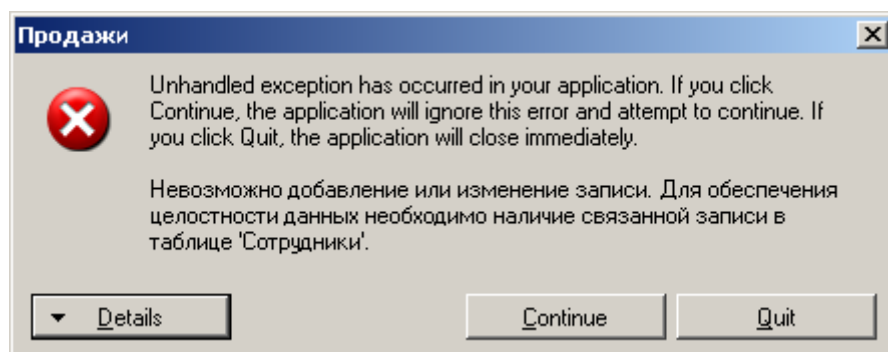
	Номер_продажи	Код_блюда	Количество	Цена	Скидка
▶	1	1	10	120,00р.	0,20
	1	2	4	140,00р.	0,00
	1	3	5	120,00р.	0,10
	1	7	2	130,00р.	0,00
*					

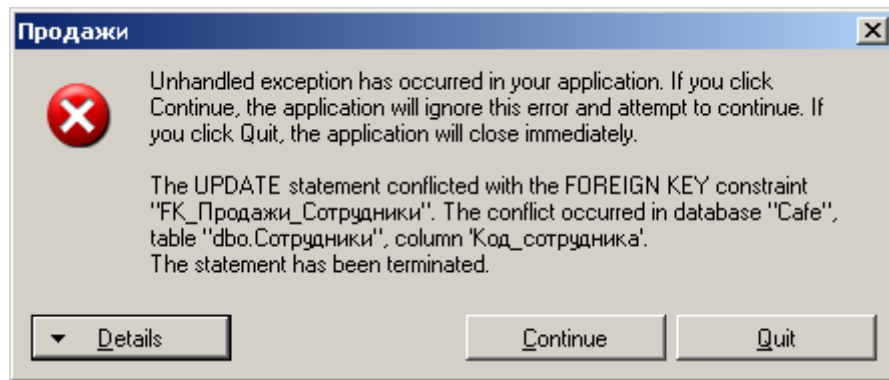
При работе с данными на данной форме могут возникнуть следующие исключения, например:

- в таблице Продажи указан повторяющийся Номер_продажи (в нашем случае – первичный ключ):

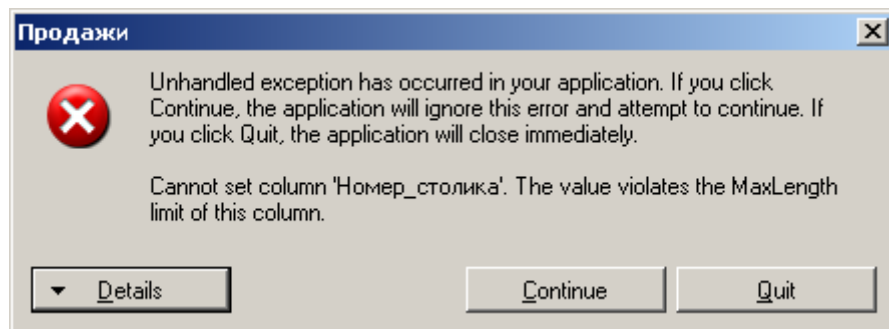


- в таблице Продажи указан Код_сотрудника, которого в БД (в родительской таблице Сотрудники) не существует (нами ранее была установлена ссылочная целостность):

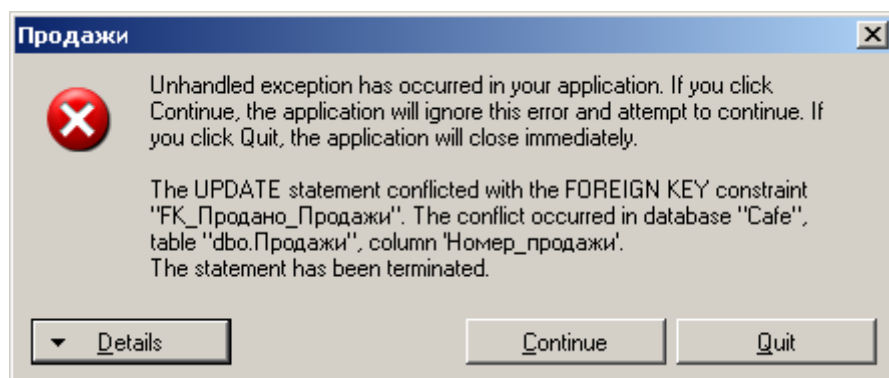
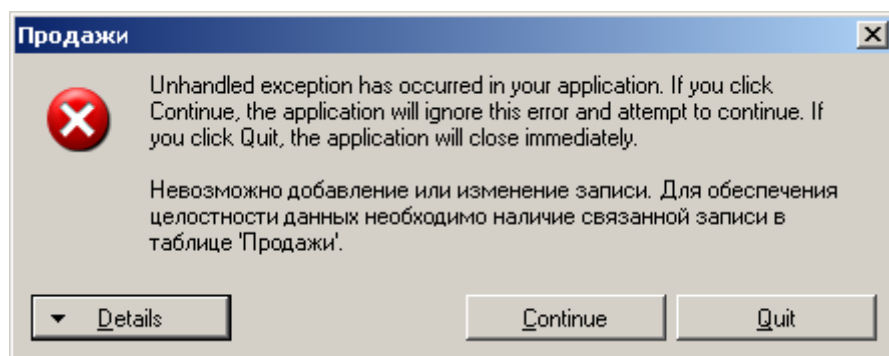




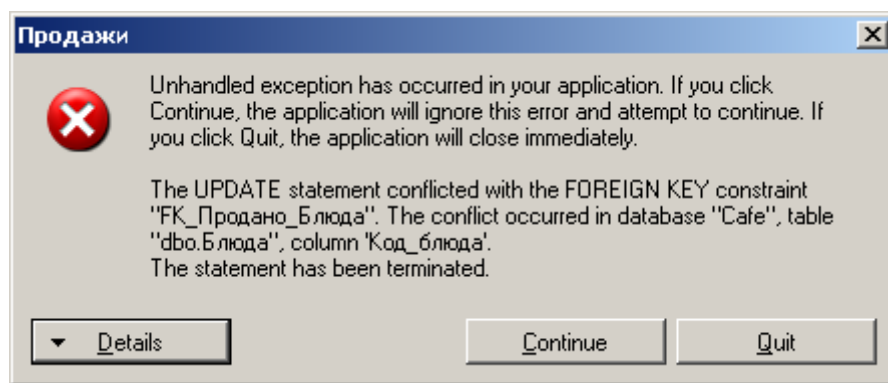
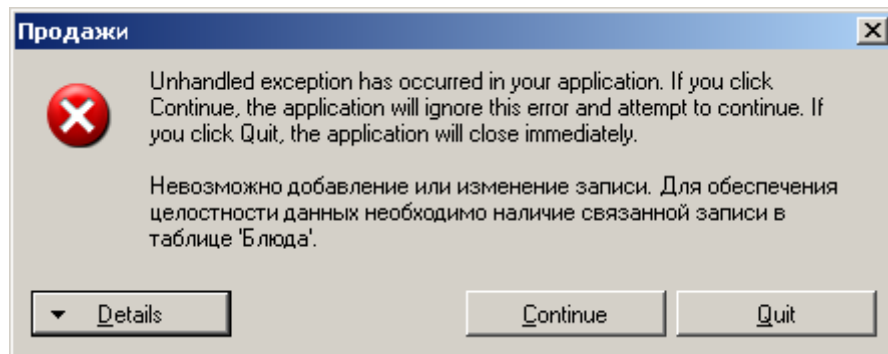
- в таблице Продажи превышено количество символов для ввода номера столика:



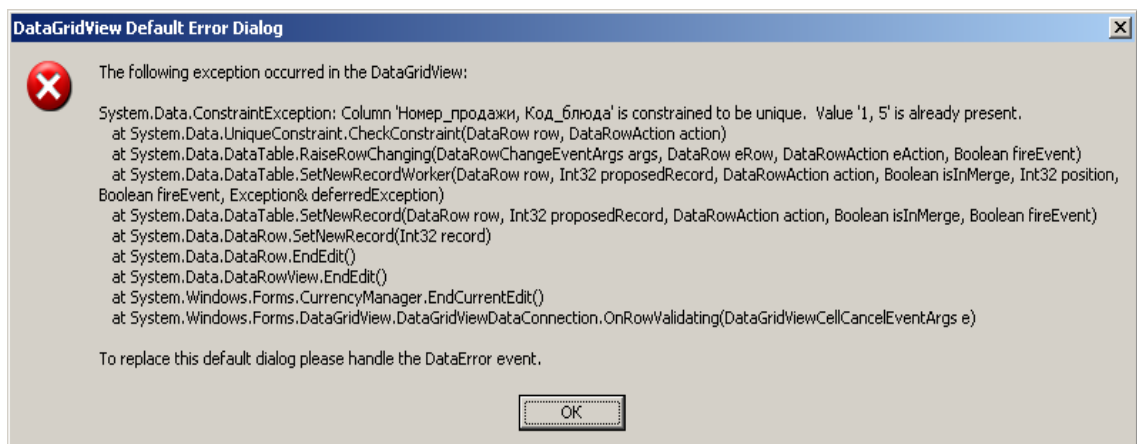
- в таблице Продано указан Номер_продажи, не существующий в БД (в родительской таблице Продажи) (нами ранее была установлена ссылочная целостность):



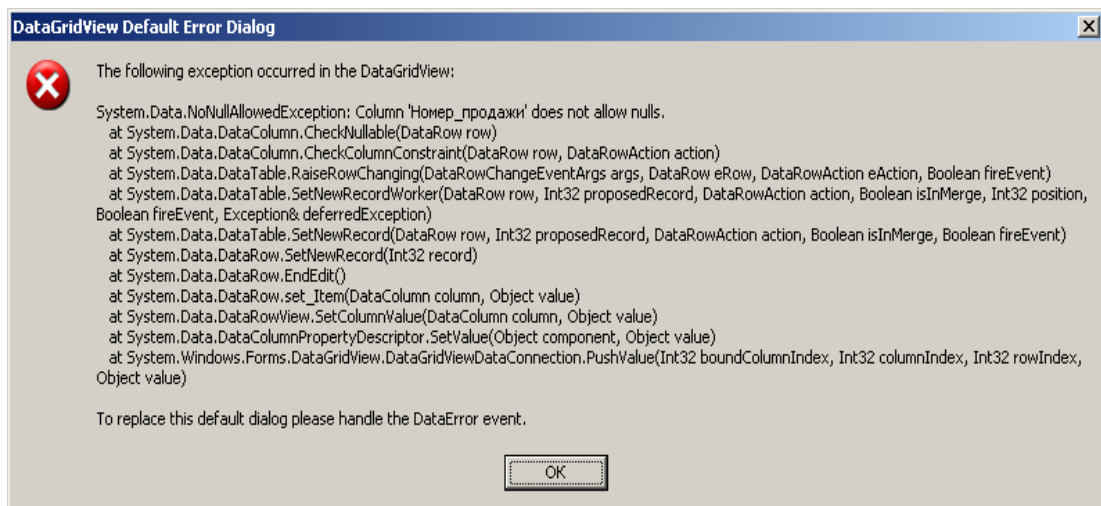
- в таблице Продано указан Код_блюда, не существующий в БД (в родительской таблице Блюда) (нами ранее была установлена ссылочная целостность):



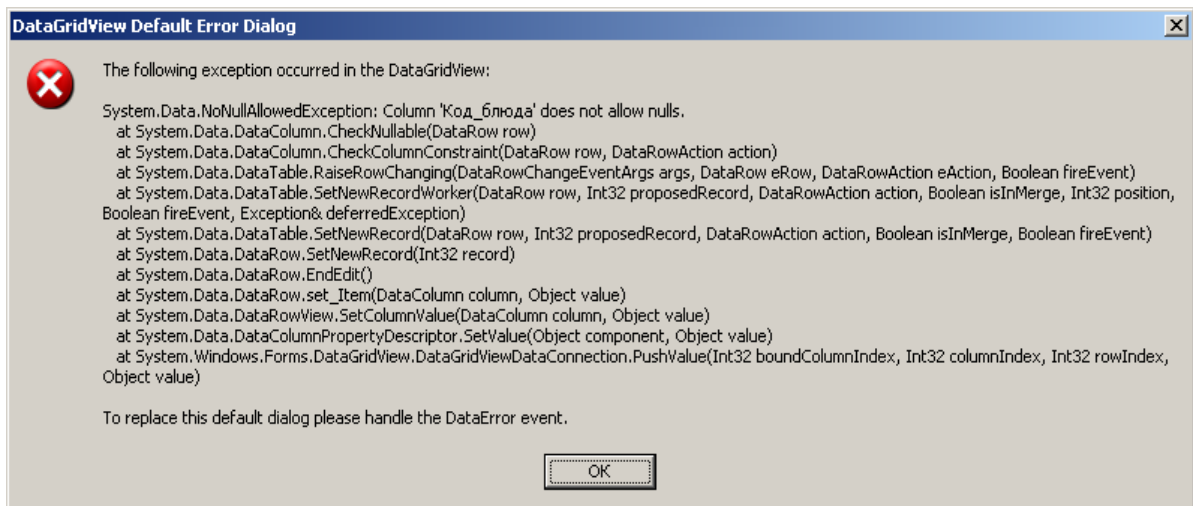
- в таблице Продано повторился первичный ключ (в нашем случае составной первичный ключ Номер_продажи и Код_блюда):



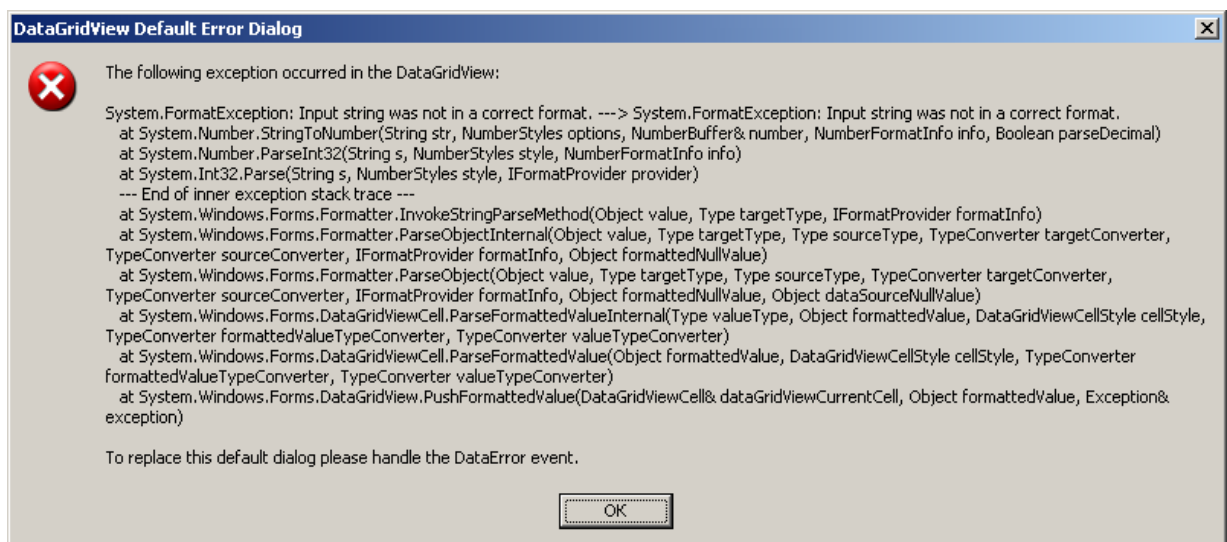
- в таблице Продано не указан Номер_продажи:



- в таблице Продано не указан Код_блюда:



- задан не тот тип данных в проданоDataGridView для
целочисленных колонок Номер_продажи, Код_блюда, Количество:



- И другие исключения согласно наложенным ограничениям в БД (см. лабораторную работу №1).

Рассмотрим как, например, можно реализовать обработку исключений при оформлении данных в таблице **Продажи** (в случае указания одинаковых номеров продаж, в продаже указан Код_сотрудника, которого в БД не существует, превышено количество символов для ввода номера столика).

Для этого в коде **FormSalesList** замените блок сохранения:

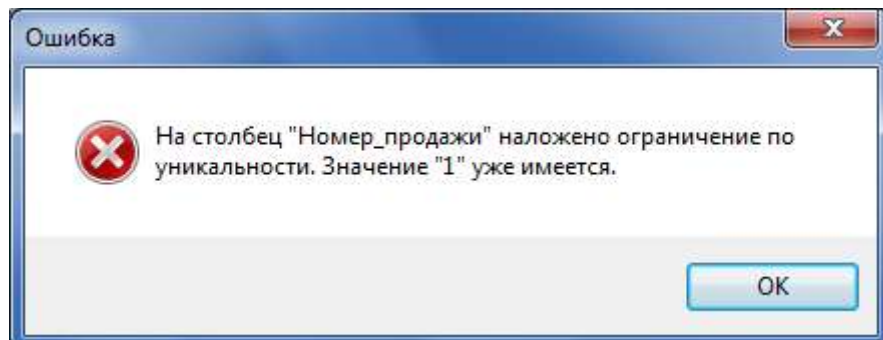
```
private void продажиBindingNavigatorSaveItem_Click(object sender, EventArgs e)
{
    this.Validate();
    this.продажиBindingSource.EndEdit();
    this.tableAdapterManager.UpdateAll(this.cafeDataSet);
}
```

На блок с конструкцией **try...catch**:

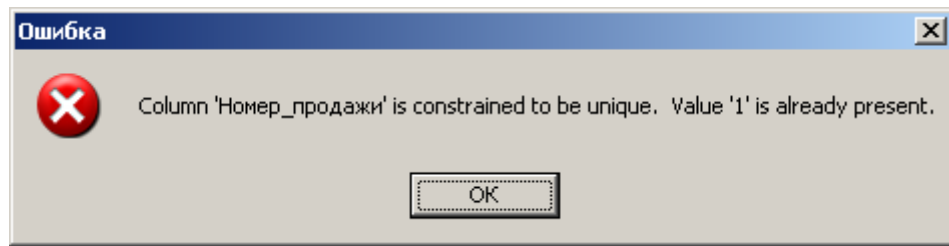
```
private void продажиBindingNavigatorSaveItem_Click(object sender, EventArgs e)
{
    try
    {
        this.Validate();
        this.продажиBindingSource.EndEdit();
        this.tableAdapterManager.UpdateAll(this.cafeDataSet);
    }
    catch (Exception err)
    {
        MessageBox.Show(err.Message, "Ошибка", MessageBoxButtons.OK,
        MessageBoxIcon.Error);
    }
}
```

Теперь будет такая обработка исключений:

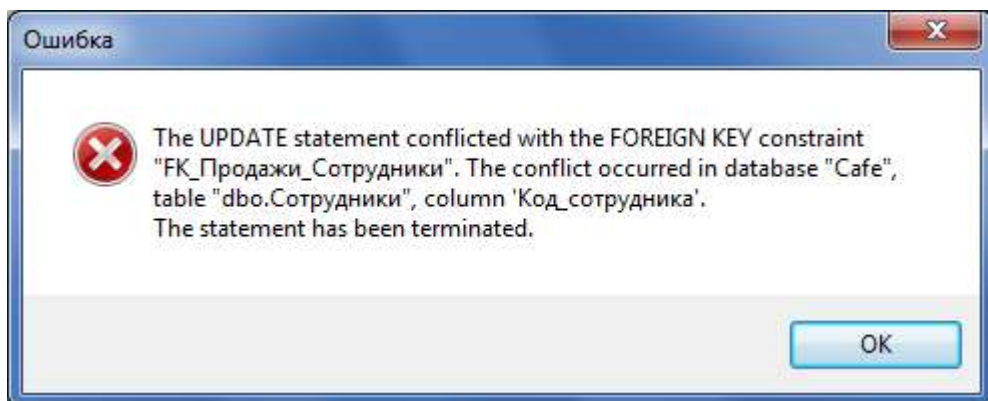
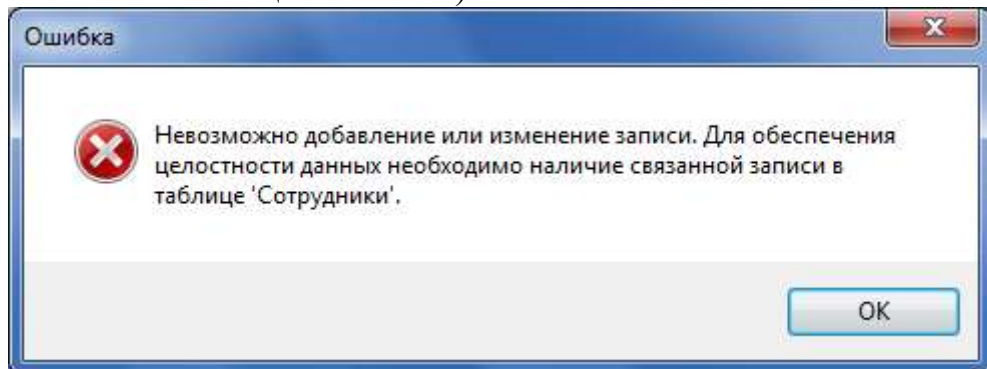
- в таблице **Продажи** указан повторяющийся **Номер_продажи** (в нашем случае – первичный ключ) (для **Microsoft Access**, **Microsoft SQL Server**):



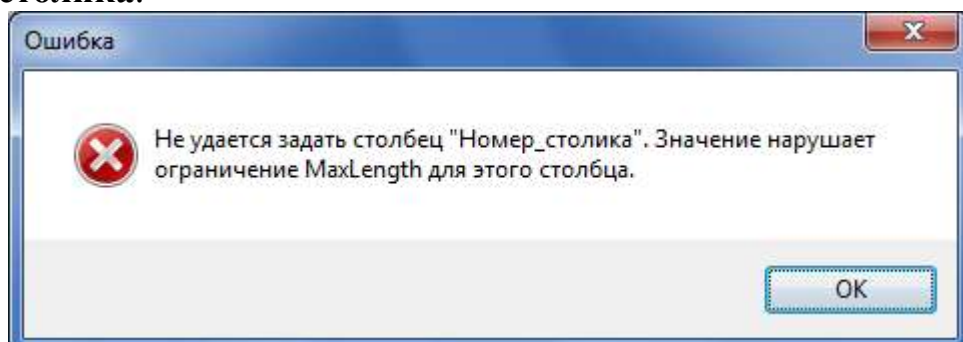
ИЛИ



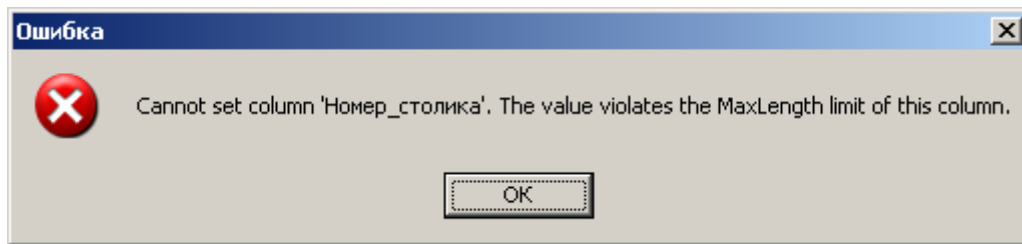
- в таблице Продажи указан Код_сотрудника, которого в БД (в родительской таблице Сотрудники) не существует (нами ранее была установлена ссылочная целостность):



- в таблице Продажи превышено количество символов для ввода номера столика:



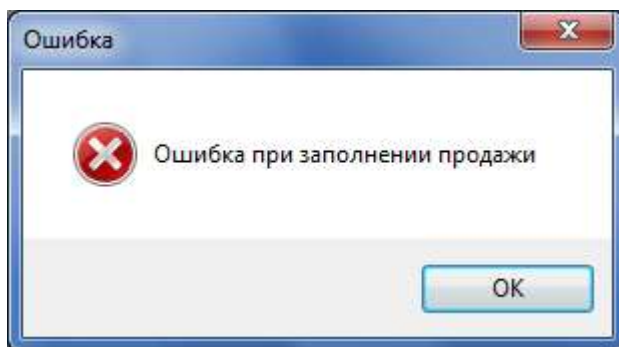
ИЛИ



Или можно задать свой текст обработки исключения:

```
private void продажиBindingNavigatorSaveItem_Click(object sender, EventArgs e)
{
    try
    {
        this.Validate();
        this.продажиBindingSource.EndEdit();
        this.tableAdapterManager.UpdateAll(this.cafeDataSet);
    }
    catch (Exception err)
    {
        MessageBox.Show("Ошибка при заполнении продажи", "Ошибка",
        MessageBoxButtons.OK, MessageBoxIcon.Error);
    }
}
```

В этом случае:



По аналогии можно обрабатывать и другие исключения.

2.3 Поиск, фильтрация данных

Для удобства пользователей в приложении необходимо предусматривать поиск/фильтрацию необходимых данных.

Поиск и фильтрацию данных можно осуществить различными способами. Например, написание SQL-запросов (см. лабораторную работу №4), использование метода Find (BindingSource.Find), свойства Filter (BindingSource.Filter) и др.

Рассмотрим как, например, можно реализовать поиск/фильтрацию с использованием метода **Find**, свойства **Filter**.

2.3.1 Метод **BindingSource.Find**

Данный метод позволяет найти нужный элемент в источнике данных. В данном методе возможны варианты:

2.3.1.1 Find(PropertyDescriptor, Object) - поиск индекса элемента по дескриптору и ключу.

Синтаксис:

public virtual int Find(PropertyDescriptor *prop*, Object *key*)

Где параметры:

***prop* (PropertyDescriptor):** тип

System.ComponentModel.PropertyDescriptor, определяет дескриптор для поиска;

***key* (Object) :** тип System.Object - искомое значение (ключ).

Возвращаемое значение: тип System .Int32 - индекс найденного элемента (начинается с нуля). Значение -1, если элемент не найден.

Этот метод обычно используется, чтобы найти первую строку, в которой значение поля, заданного дескриптором, равно значению параметра ключа.

Например:

```
int indexPos = сотрудникиBindingSource.Find("Фамилия", textBox1.Text);
if (indexPos > -1)
    сотрудникиBindingSource.Position = indexPos;
else
    MessageBox.Show("Таких сотрудников нет");
```

2.3.1.2 Find(String, Object) - поиск индекса элемента по имени и значению.

Синтаксис:

public int Find(string *propertyName*, Object *key*)

Где параметры:

***propertyName*:** тип System.String - имя свойства для поиска;

***key*:** тип System.Object - искомое значение (ключ).

Возвращаемое значение: тип System .Int32 - индекс найденного элемента (начинается с нуля). Значение -1, если элемент не найден.

Например:

```
int indexPos = сотрудникиBindingSource.Find("Фамилия", "Петров");
сотрудникиBindingSource.Position = indexPos;
```

2.3.2 Свойство **BindingSource.Filter**

Данное свойство задает выражение, используемое для фильтрации строк.

Тип для данного свойства: **System .String** - строка, которая определяет, как строки необходимо фильтровать.

Для формирования значения фильтра необходимо указать имя столбца, затем оператор и значения для фильтра.

Если основным источником данных DataSet, DataTable или DataView, можно указать логические выражения с помощью синтаксиса свойства DataColumn.Expression.

Например:

```
сотрудникиBindingSource.Filter = "Фамилия = 'Петров'"
```

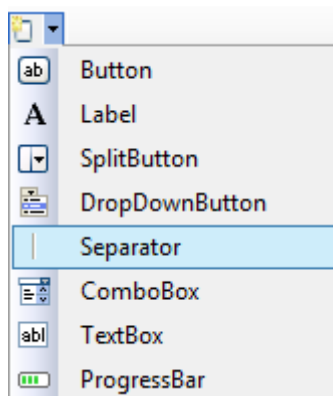
2.3.3 Реализация поиска и фильтрации данных

Реализуем, например, поиск/фильтрацию данных необходимого сотрудника. Данная возможность пригодится пользователям, работающим с данными по кадровому персоналу кафе, а также пользователям, оформляющим продажи для поиска необходимо кода сотрудника.

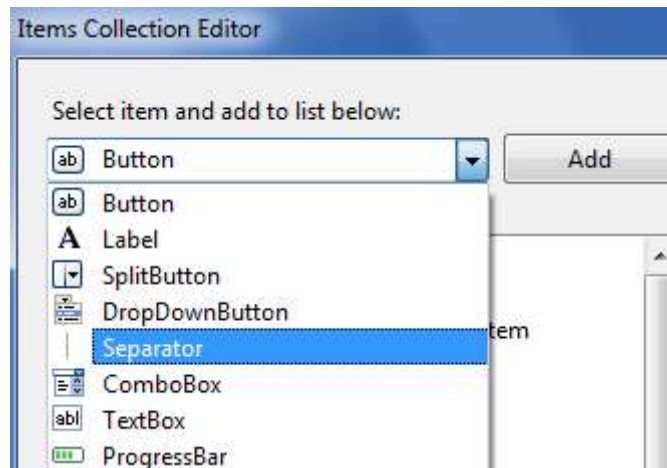
Будем осуществлять поиск/фильтр заданного в строке ввода значения по выбранной пользователем в **сотрудникиDataGridView** колонке.

Для этого перейдите на форму **FormWorkesList**.

На панели **сотрудникиBindingNavigator** рядом с кнопкой **Сохранить** (**сотрудникиBindingNavigatorSaveItem**) разместите разделитель (**Separator**) через **Add ToolStripButton**:



Или через **Edit Items**:

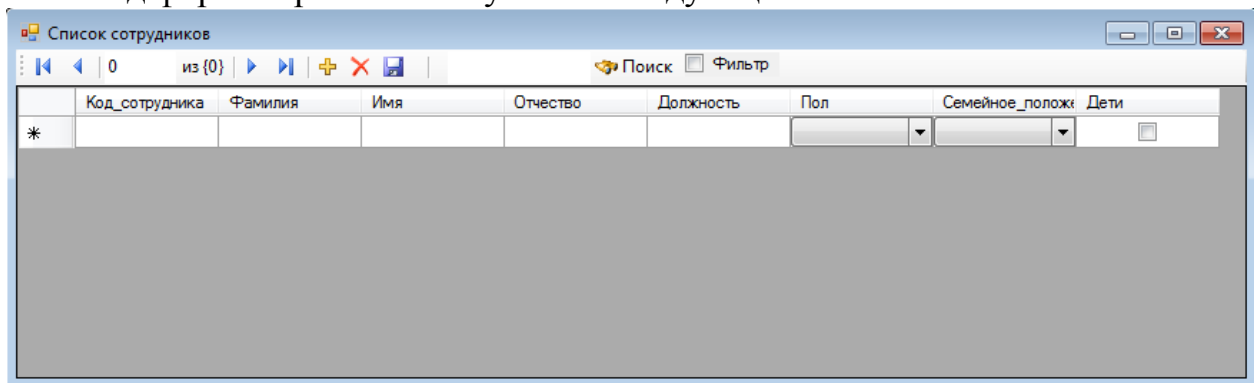


Рядом строку ввода **TextBox** для которой укажите:
 - свойство **Name** в **toolStripTextBoxFind**.

Рядом с ней кнопку **Button** для поиска для которой укажите:
 - свойство **Name** в **toolStripButtonFind**;
 - свойство **DisplayStyle** в **ImageAndText**;
 - свойство **Text** в **Поиск**;
 - свойство **TextAlign** в **MiddleRight**;
 - свойство **Image** в соответствующую картинку;
 - свойство **ImageAlign** в **MiddleLeft**.

Рядом с ней компонент **CheckBox** с **Toolbox** для которого укажите:
 - свойство **Name** в **checkBoxFind**;
 - свойство **Text** в **Фильтр**.

Вид формы при этом получается следующий:



Как было сказано ранее, будем осуществлять поиск заданного в строке ввода **toolStripTextBoxFind** значения по выбранной пользователем в **сотрудникиDataGridView** колонке.

Пропишем обработчик для отбора выбранной пользователем колонки в **сотрудникиDataGridView**.

Для этого в коде **FormWorkesList** пропишите:

```

string GetSelectedFieldName()
{
    return
сотрудникиDataGridView.Columns[сотрудникиDataGridView.CurrentCell.ColumnIndex
].DataPropertyName;
}

```

На обработчике события **Click** кнопки **Поиск (toolStripButtonFind)** пропишите:

```

private void toolStripButtonFind_Click(object sender, EventArgs e)
{
    if (toolStripTextBoxFind.Text == "")
    {
        MessageBox.Show("Вы ничего не задали", "Внимание",
MessageBoxButtons.OK, MessageBoxIcon.Information);
        return;
    }

    int indexPos;

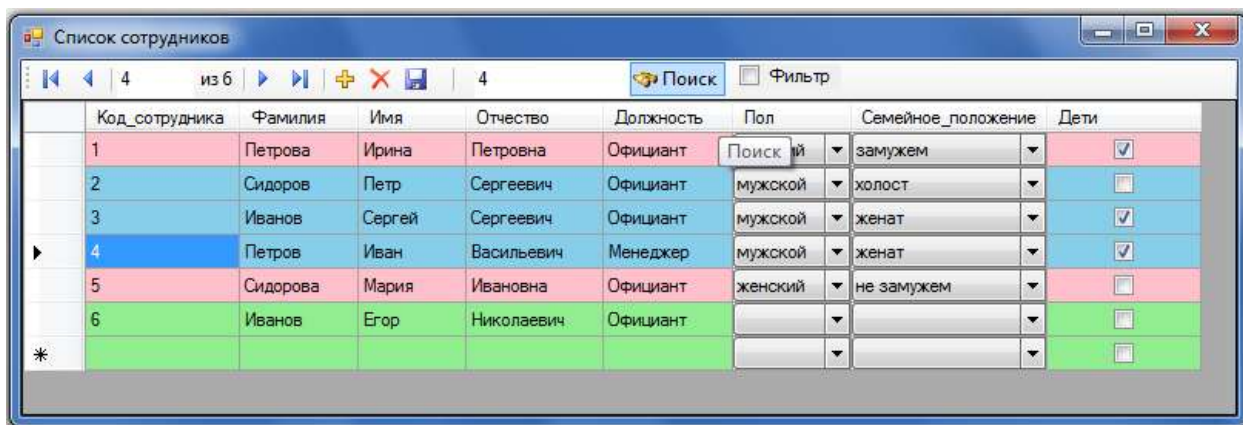
    try
    {
        indexPos =
сотрудникиBindingSource.Find(GetSelectedFieldName(),
toolStripTextBoxFind.Text);
    }
    catch (Exception err)
    {
        MessageBox.Show("Ошибка поиска \n" + err.Message);
        return;
    }

    if (indexPos > -1)
        сотрудникиBindingSource.Position = indexPos;
    else
    {
        MessageBox.Show("Таких сотрудников нет", "Внимание",
MessageBoxButtons.OK, MessageBoxIcon.Information);
        сотрудникиBindingSource.Position = 0;
    }
}

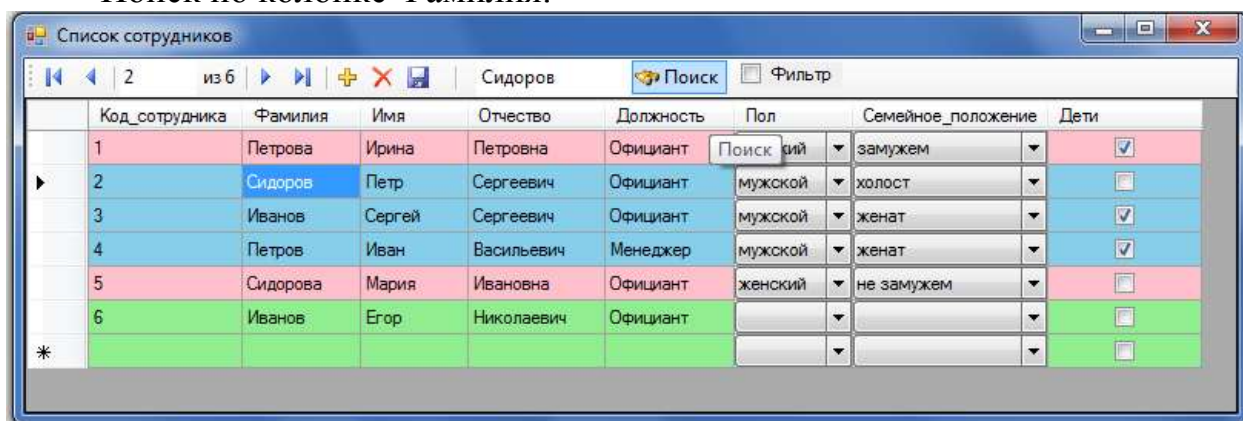
```

Запустите программу. Посмотрите, как обрабатывает поиск данных, например:

Поиск по колонке Код_сотрудника:



Поиск по колонке Фамилия:



Закройте программу. Вернитесь в проект.

На обработчике **checkBoxFind** для **фильтрации данных** пропишите:

```
private void checkBoxFind_CheckedChanged(object sender, EventArgs e)
{
    if (checkBoxFind.Checked)
    {
        if (toolStripTextBoxFind.Text == "")
            MessageBox.Show("Вы ничего не задали", "Внимание",
                MessageBoxButtons.OK, MessageBoxIcon.Information);
        else
            try
            {
                сотрудникиBindingSource.Filter =
                    GetSelectedFieldName() + "='" + toolStripTextBoxFind.Text + "'";
            }
            catch (Exception err)
            {
                MessageBox.Show("Ошибка фильтрации \n" +
                    err.Message);
            }
    }
    else
        сотрудникиBindingSource.Filter = "";

    if (сотрудникиBindingSource.Count == 0)
    {
        MessageBox.Show("Нет таких");
    }
}
```



```

        сотрудникиBindingSource.Filter = "";
        checkBoxFind.Checked = false;
    }

}

```

Запустите программу. Посмотрите, как отрабатывает фильтрация данных, например:

Фильтр по фамилии Сидоров:

Список сотрудников

1 из 1

Сидоров Поиск Фильтр

	Код_сотрудника	Фамилия	Имя	Отчество	Должность	Пол	Семейное_положение	Дети
▶	2	Сидоров	Петр	Сергеевич	Официант	мужской	холост	☐
*								☐

Фильтр по должности официант:

Список сотрудников

2 из 5

официант Поиск Фильтр

	Код_сотрудника	Фамилия	Имя	Отчество	Должность	Пол	Семейное_положение	Дети
	1	Петрова	Ирина	Петровна	Официант	женский	замужем	☒
▶	2	Сидоров	Петр	Сергеевич	Официант	мужской	холост	☐
	3	Иванов	Сергей	Сергеевич	Официант	мужской	женат	☒
	5	Сидорова	Мария	Ивановна	Официант	женский	не замужем	☐
	6	Иванов	Егор	Николаевич	Официант			☐
*								☐

Закройте программу. Вернитесь в проект.

Для удобства пользователей по аналогии реализуются поиск/фильтрация данных по блюдам, продажам.

2.4 Связь 1:М между Сотрудники и Продажи в приложении

Как было сказано ранее, в БД кафе таблица **Сотрудники** связана с таблицей **Продажи** связью 1:М.

Рассмотрим как можно, например, реализовать данную связь в приложении.

При работе с продажами не всегда удобно помнить код необходимого сотрудника.

Кроме того, следует отметить, что нами была установлена ссылочная целостность в БД, т.е. должен быть указан код существующего в БД сотрудника, иначе будет ошибка при работе с данными (см. раздел 2.2).

В нашем случае, чтобы посмотреть код необходимого сотрудника необходимо перейти на форму Сотрудники и посмотреть для него код.

Данную ситуацию можно изменить. Например, можно организовать вызов формы с сотрудниками из формы Продажи.

Для этого перейдите на форму Продажи (**FormSalesList**).

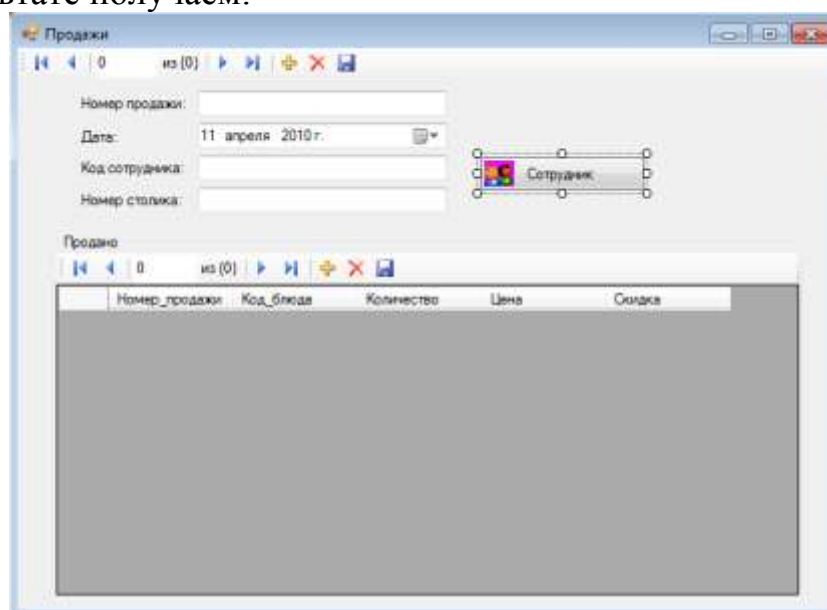
Рядом с **код_сотрудникаTextBox** разместите с панели инструментов **Toolbox** кнопку **button**.

Укажите для **button**:

- свойство **Name** в **buttonWorker**;
- свойство **Text** в **Сотрудник**.

При желании данной кнопке можно указать необходимую картинку через свойство **Image**. А через свойство **ImageAlign** можно указать в каком месте данную картинку на этой кнопке выводить, например, для слева по центру укажите **MiddleLeft**.

В результате получаем:



По нажатию на данную кнопку **buttonWorker (Сотрудник)** будем показывать форму с данными по сотрудникам. В данной форме пользователь выбирает необходимого ему сотрудника и его данные (код сотрудника) заносятся в соответствующую продажу.

Следует напомнить, что таблицы **Сотрудники** и **Продажи** связаны связью 1:М по колонке Код_сотрудника.

Когда пользователь будет выбирать сотрудника в форме Сотрудники, необходимо запомнить код выбранного сотрудника и передать его в

соответствующее поле таблицы **Продажи** Код_сотрудника (реализовать связь между **Сотрудники** и **Продажи** (1:M) по **Код_сотрудника**).

Перейдите на форму **Сотрудники** (**FormWorkesList**).

Чтобы пользователю было удобней выбирать сотрудника, добавьте на форме **FormWorkesList** на панели **сотрудникиBindingNavigator** кнопку **Button** для которой укажите:

- свойство **Name** в **toolStripButtonOK**;
- свойство **DisplayStyle** в **ImageAndText**;
- свойство **Text** в **Выбрать**;
- свойство **TextAlign** в **MiddleRight**;
- свойство **Image** в соответствующую картинку;
- свойство **ImageAlign** в **MiddleLeft**;
- в событии **Click** укажите:

```
private void toolStripButtonOK_Click(object sender, EventArgs e)
{
    DialogResult = DialogResult.OK;
}
```

Примечание:

Если Вами была добавлена кнопка **Button** с панели инструментов **Toolbox**, то для нее можно непосредственно через **Properties** свойство **DialogResult** указать в **OK**.

Реализуем следующую логику:

если пользователь нажал кнопку **Выбрать** (**toolStripButtonOK**), то запоминаем код выбранного сотрудника. Если он закрыл форму, нажав , то код сотрудника не запоминаем, считая, что пользователь передумал выбирать сотрудника.

Кнопку **Выбрать** (**toolStripButtonOK**) пользователь видит только когда открывает список сотрудников из формы **Продажи**.

Реализуем придуманную логику.

В коде формы **FormWorkesList** пропишите:

```
//переменная для текущего(выбранного) кода сотрудника
int idCurrent = -1;

public int ShowSelectForm(int id)
{
```

```

        toolStripButtonOK.Visible = true;
        idCurrent = id;
        if (ShowDialog() == DialogResult.OK)
            return
    (int) ((DataRowView) сотрудникиBindingSource.Current) ["код_сотрудника"];
        else
            return -1;
    }

```

Перейдите на форму **FormSalesList**.

В коде **FormSalesList** пропишите обработчик нажатия кнопки **Сотрудник (buttonWorker)** :

```

private void buttonWorker_Click(object sender, EventArgs e)
{
    int id = -1;
    if
    (((DataRowView) продажиBindingSource.Current) ["код_сотрудника"].ToString() !=
    "")
    {
        id =
    (int) ((DataRowView) продажиBindingSource.Current) ["код_сотрудника"];
    }
    id = FormWorkesList.fw.ShowSelectForm(id);
    if (id >= 0)
    {
        MessageBox.Show(id.ToString());
        ((DataRowView) продажиBindingSource.Current) ["код_сотрудника"]
= id;
        продажиBindingSource.EndEdit();
    }
}

```

Запустите программу. Посмотрите полученную возможность.

Обратите внимание, что при открытии формы Сотрудники по кнопке **buttonWorker** курсор уснавливается на первую запись, а не на текущего сотрудника в данной продаже.

Вернитесь в проект, чтобы эту ситуацию изменить.

Перейдите на форму **FormWorkesList**.

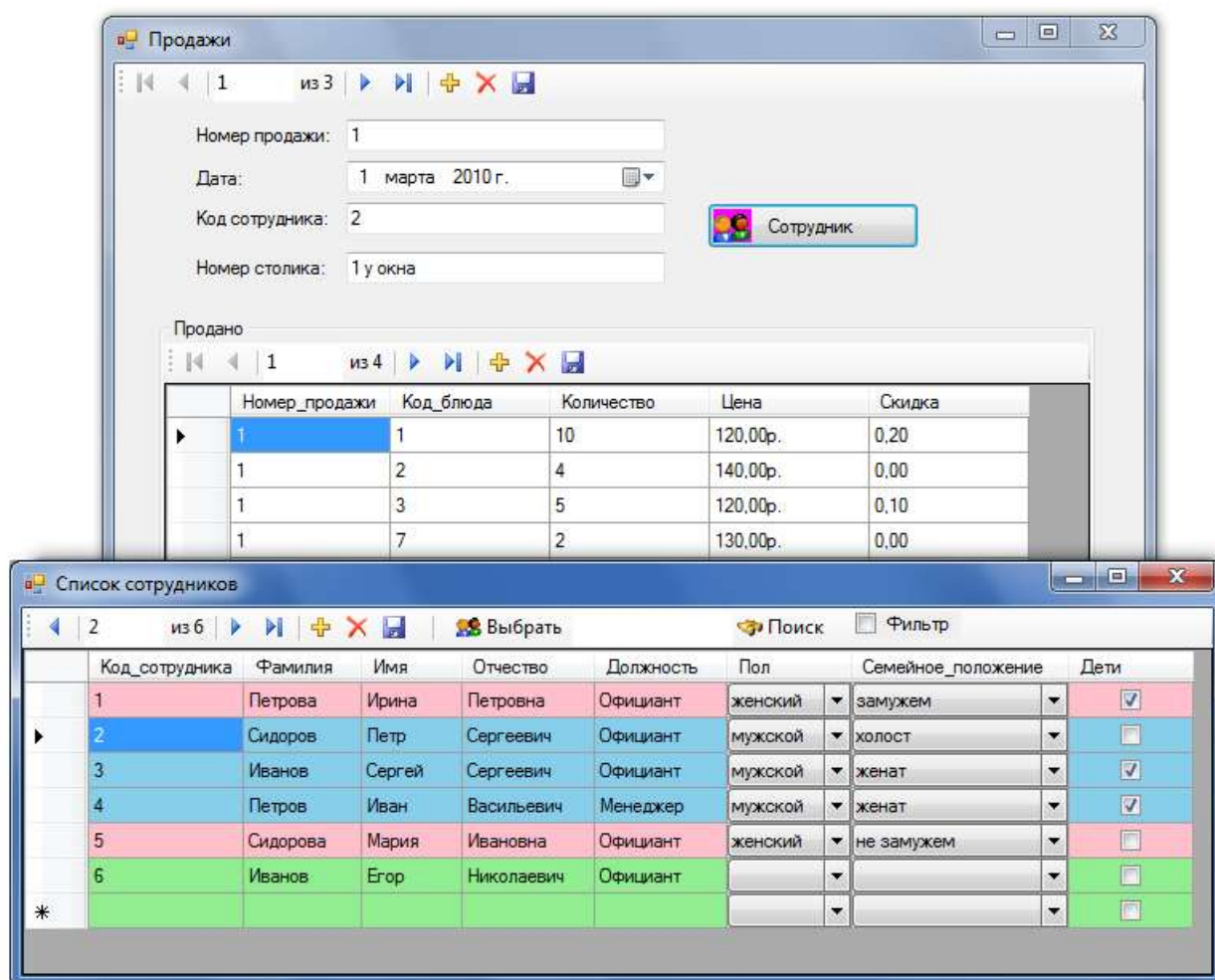
В событии **Shown** для нее пропишите установку позиции на текущего сотрудника в продаже:

```

private void FormWorkesList_Shown(object sender, EventArgs e)
{
    сотрудникиBindingSource.Position =
    сотрудникиBindingSource.Find("код_сотрудника", idCurrent);
}

```

Запустите программу. Посмотрите, что при открытии формы Сотрудники курсор устанавливается на заданного сотрудника в Продажах:



При открытии же справочника сотрудников не из формы для продаж, в сотрудниках не нужно устанавливать курсор на текущего в продаже сотрудника, а также необходимо спрятать кнопку **Выбрать** (toolStripButtonOK).

Для этого измените код для метода **ShowForm()** на форме **FormWorkesList**:

```
public void ShowForm()
{
    toolStripButtonOK.Visible = false;
    сотрудникиBindingSource.Position = 0;
    Show();
    Activate();
}
```

Теперь позиция будет на первой записи при открытии справочника сотрудников не через форму Продажи, например, через пункты меню.

Примечание:

Помимо формы можно использовать и другие способы взятия информации из таблицы **Сотрудники в Продажи**, например, использование **ListBox**, **ComboBox** и др.

2.5 Создание вычисляемых колонок

Таблицы могут содержать колонки, значения которых получаются расчетным путем. Например, в записях элементов **Продано** имеет смысл вычислять колонку **Стоимость** (произведение количества на цену с учетом скидки). Вычисляемые колонки могут также содержаться в дочерних записях.

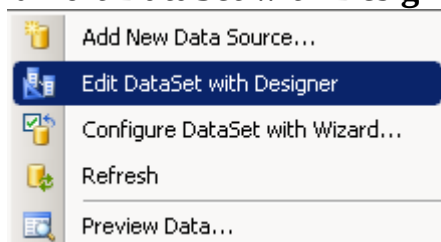
Для создания вычисляемых колонок используются выражения (**Expression**).

Синтаксис выражений состоит из стандартных арифметических операций, булевых и строковых операторов, а также констант. Вы можете сослаться на значения некоторых данных, используя имена колонок и таблиц (как в SQL-запросах), включать агрегирующие функции.

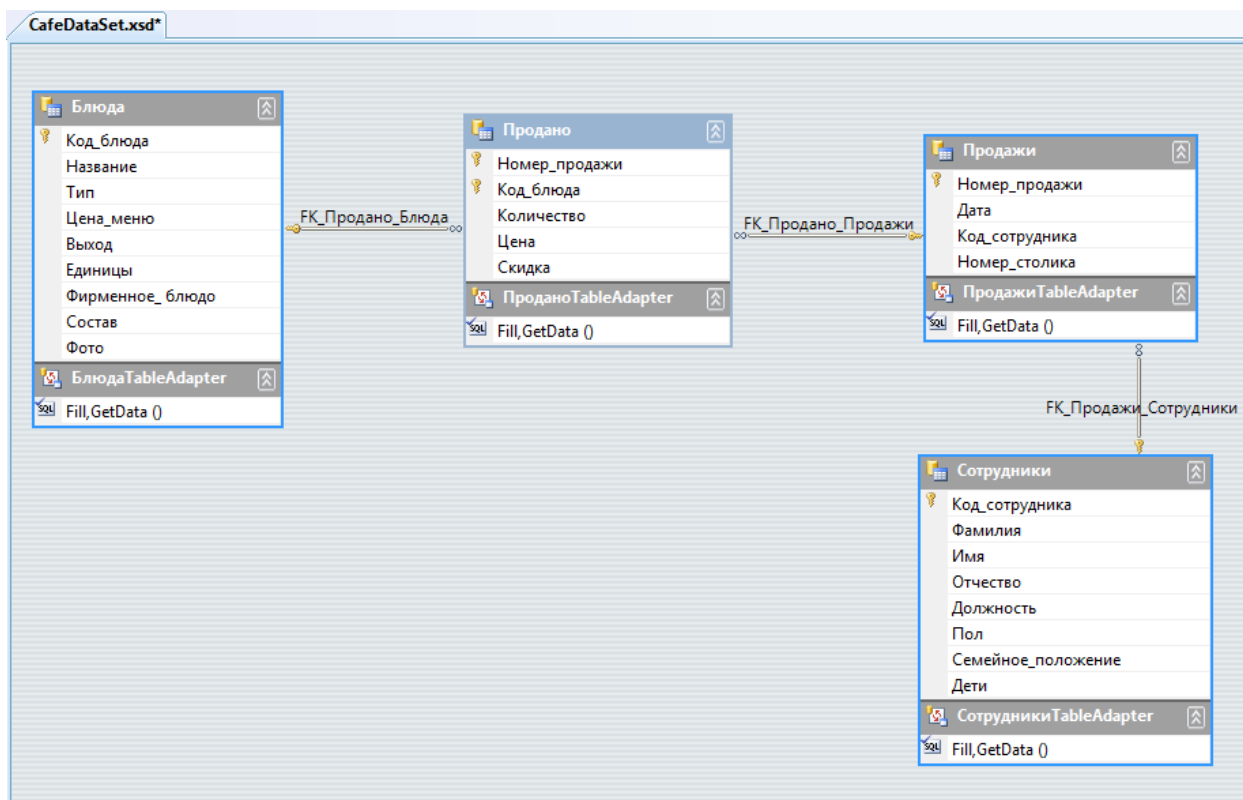
Вычисляемые колонки можно создавать и через непосредственно SQL запрос (см. лабораторную работу №4).

Для создания вычисляемых колонок в наборе данных необходимо выполнить следующие шаги.

Если схема еще не открыта в дизайнере, щелкните правой кнопкой на наборе данных и выберите **Edit DataSet with Designer**:

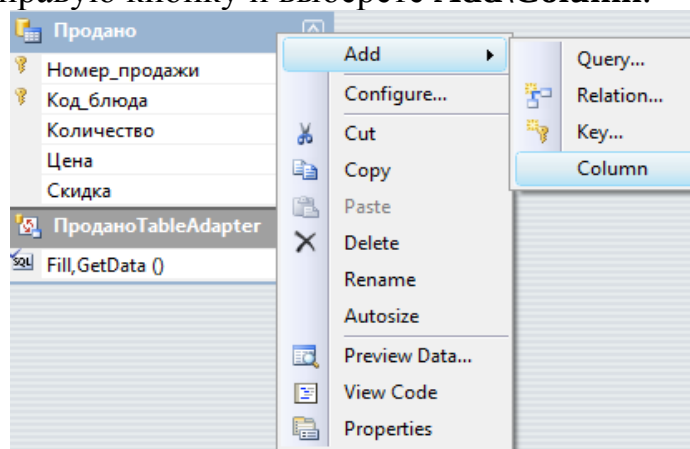


Или два раза щелкните мышью на XML-схеме (xsd-файл) в **Solution Explorer**. В нашем случае на файле **CafeDataSet.xsd**.

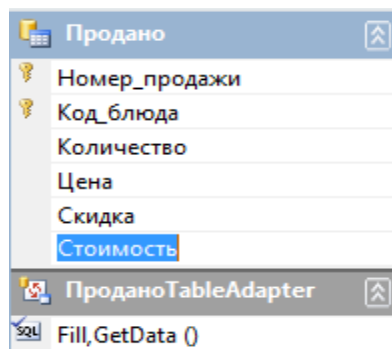


В сетке, представляющей таблицу, с которой вы хотите работать, добавьте новую колонку, выполняя следующие шаги:

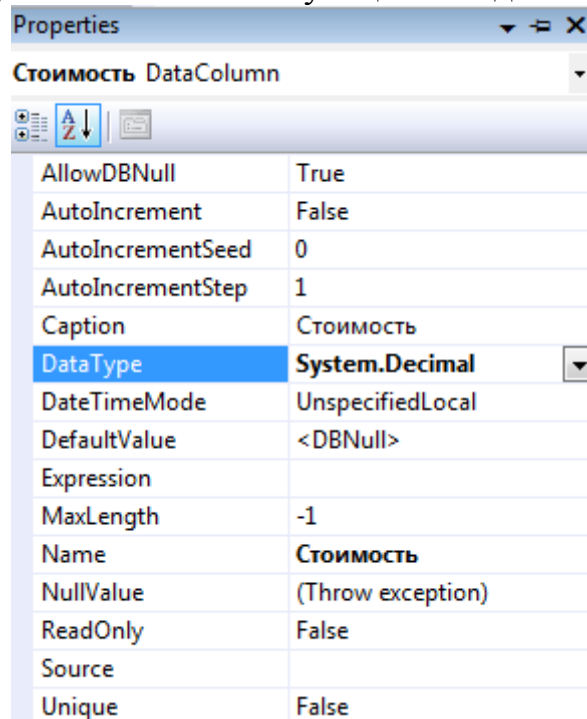
- нажмите правую кнопку и выберите **Add\Column**:



- введите имя колонки, например, **Стоимость**;



- в свойствах укажите соответствующий тип данных:



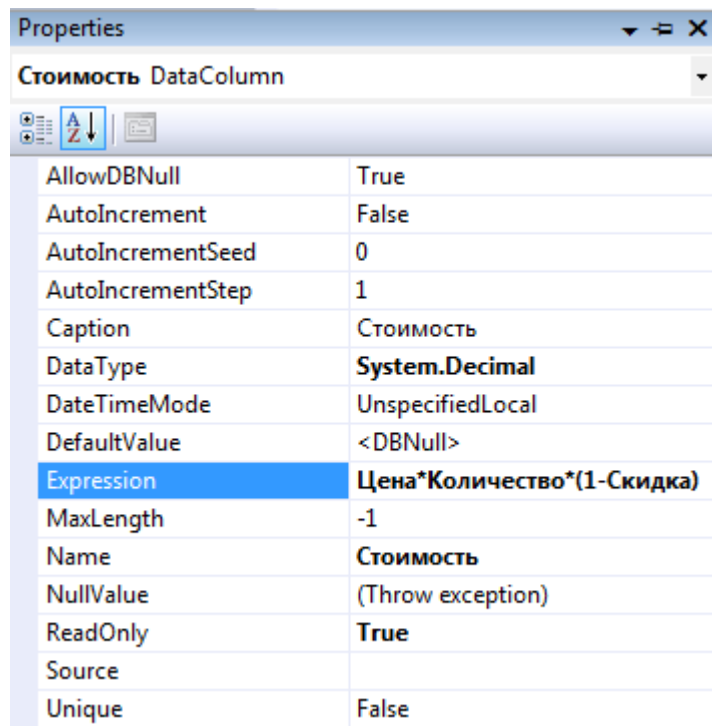
Properties

Стоимость DataColumn

AllowDBNull	True
AutoIncrement	False
AutoIncrementSeed	0
AutoIncrementStep	1
Caption	Стоимость
DataType	System.Decimal
DateTimeMode	UnspecifiedLocal
DefaultValue	<DBNull>
Expression	
MaxLength	-1
Name	Стоимость
NullValue	(Throw exception)
ReadOnly	False
Source	
Unique	False

- оставаясь на добавленной колонке в сетке таблицы, перейдите в окно **Properties** и найдите там свойство **Expression**. В данной строке введите значение вычисляемого выражения, например, в данном случае **Стоимость** вычисляется с учетом скидки:

Цена*Количество*(1-Скидка)



Properties

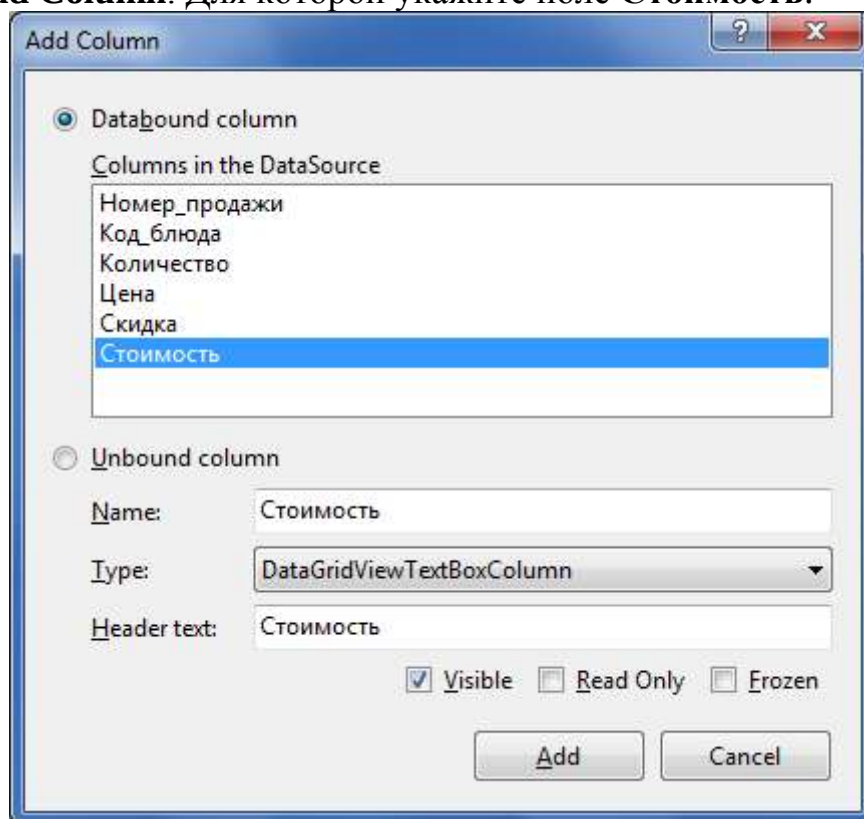
Стоимость DataColumn

AllowDBNull	True
AutoIncrement	False
AutoIncrementSeed	0
AutoIncrementStep	1
Caption	Стоимость
DataType	System.Decimal
DateTimeMode	UnspecifiedLocal
DefaultValue	<DBNull>
Expression	Цена*Количество*(1-Скидка)
MaxLength	-1
Name	Стоимость
NullValue	(Throw exception)
ReadOnly	True
Source	
Unique	False

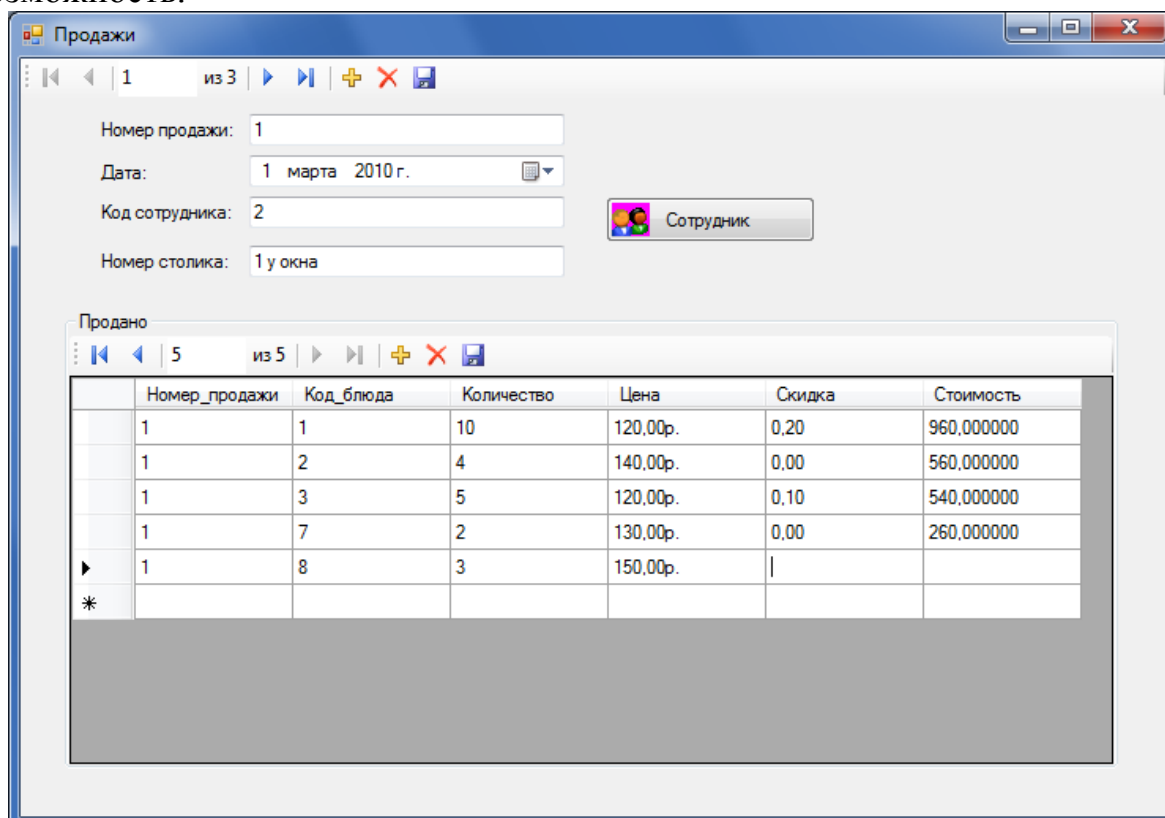
Перейдите на форму **FormSalesList**.

Теперь настроим **проданоDataGridView**, чтобы он отобразил новое поле **Стоимость**.

Для этого нажмите правую кнопку на **проданоSaleDataGridView** и укажите **Add Column**. Для которой укажите поле **Стоимость**:



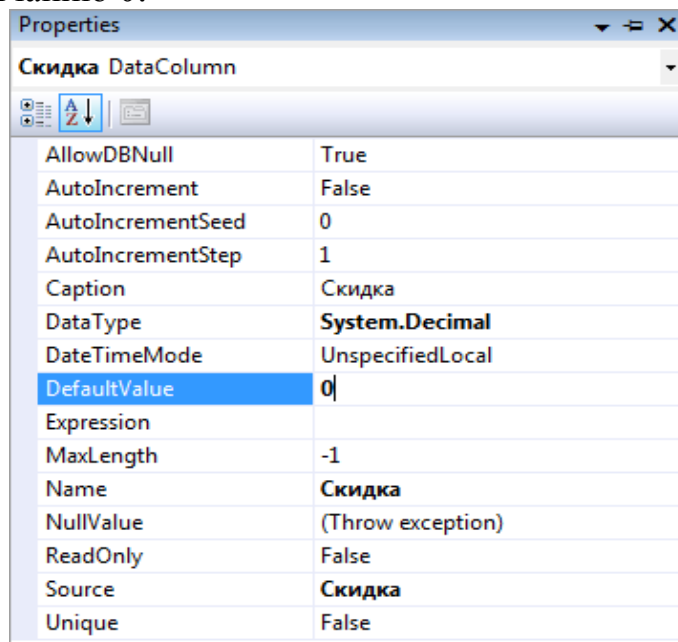
Настройте размеры формы, чтобы можно было увидеть данную колонку в приложении. Запустите программу и посмотрите настроенную возможность.



Обратите внимание, что если скидка не указана, т.е. содержит Null значение, то стоимость не вычисляется.

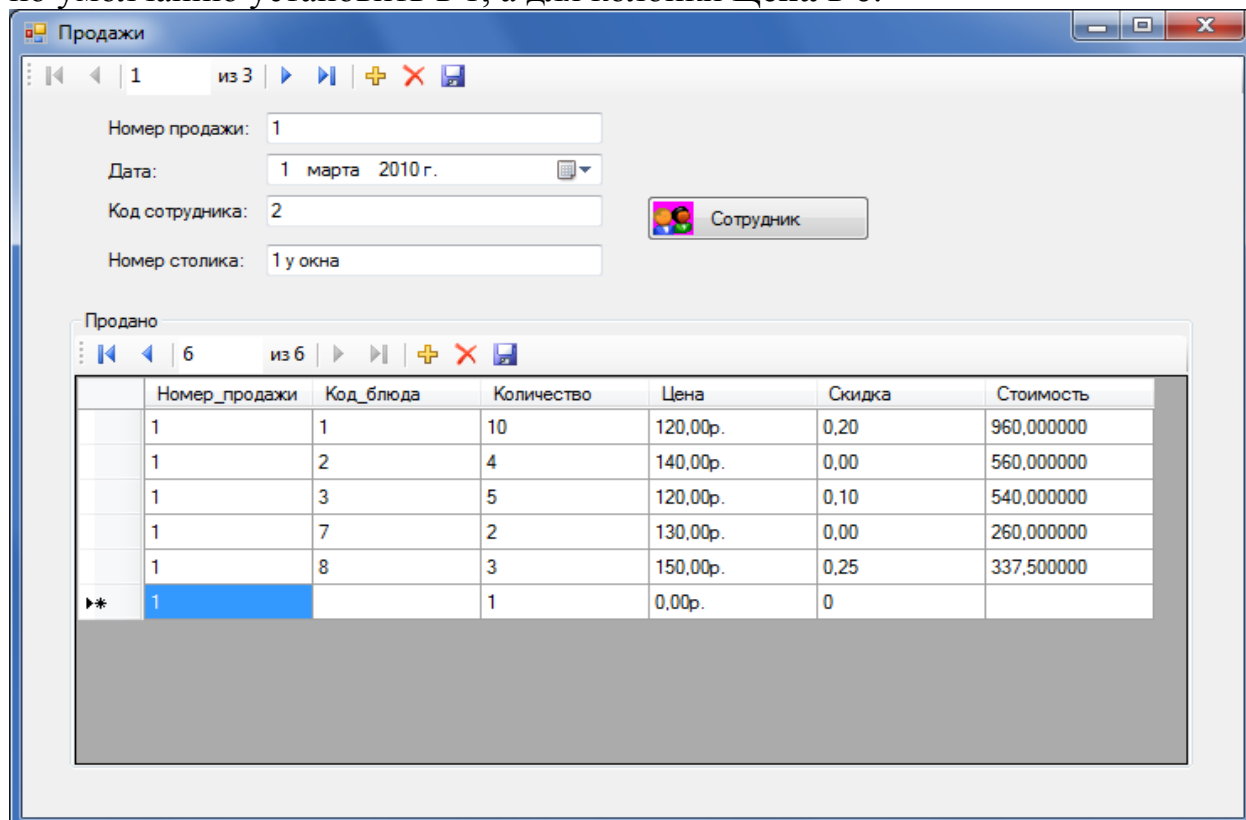
Чтобы этого избежать, необходимо для колонки **Скидка** установить значение по умолчанию 0.

Для этого в дизайнера набора данных укажите для поля **Скидка** значение по умолчанию 0:



Теперь при работе с данными значение по умолчанию для скидки равно 0.

По аналогии можно для удобства для колонки **Количество** значение по умолчанию установить в 1, а для колонки **Цена** в 0.



По аналогии с колонкой **Цена** укажите для колонки **Стоимость** денежный формат с точностью в 2 знака (см. раздел 2.1).

Запустите программу. Посмотрите настроенную возможность.

The screenshot shows a window titled "Продажи" (Sales). It contains a form with the following fields:

- Номер продажи: 1
- Дата: 1 марта 2010 г.
- Код сотрудника: 2
- Номер столика: 1 у окна

Below the form is a button labeled "Сотрудник" (Employee) with a small icon. Underneath is a section titled "Продано" (Sold) containing a table with 7 columns: "Номер_продажи", "Код_блюда", "Количество", "Цена", "Скидка", and "Стоимость".

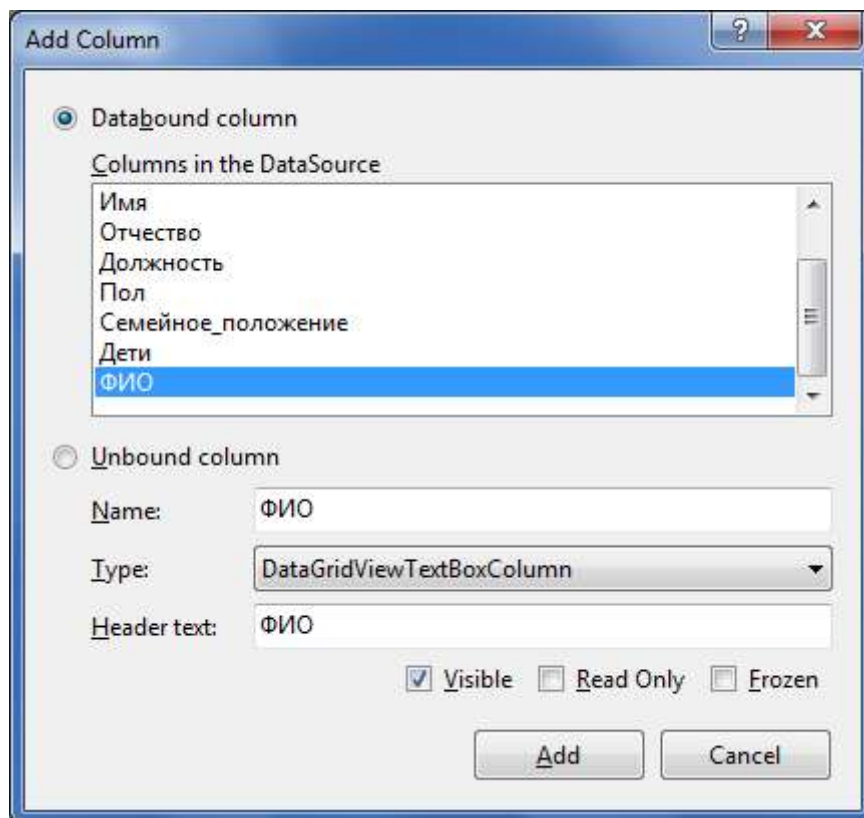
	Номер_продажи	Код_блюда	Количество	Цена	Скидка	Стоимость
▶	1	1	10	120,00р.	0,20	960,00р.
	1	2	4	140,00р.	0,00	560,00р.
	1	3	5	120,00р.	0,10	540,00р.
	1	7	2	130,00р.	0,00	260,00р.
	1	8	3	150,00р.	0,25	337,50р.
*						

Аналогично создайте вычисляемую колонку **ФИО** строкового типа в таблице **Сотрудники**:

The screenshot shows the "Properties" window for a "ФИО DataColumn". The properties are as follows:

Property	Value
AllowDBNull	True
AutoIncrement	False
AutoIncrementSeed	0
AutoIncrementStep	1
Caption	ФИО
DataType	System.String
DateTimeMode	UnspecifiedLocal
DefaultValue	<DBNull>
Expression	Код_сотрудника+' '+Фамилия+' '+Имя+' '+Отчество
MaxLength	-1
Name	ФИО
NullValue	(Throw exception)
ReadOnly	True
Source	
Unique	False

По аналогии укажите данное поле в **сотрудникиDataGridView**:



Настройте, чтобы в приложении была видна полученная колонка.

Запустите программу, посмотрите отработку вычисляемого поля ФИО:

На рисунке показана форма списка сотрудников, открытая не через форму продаж:

The screenshot shows the 'Список сотрудников' application window. It features a data grid with the following columns: 'Код_сотрудника', 'Фамилия', 'Имя', 'Отчество', 'Должность', 'Пол', 'Семейное_положение', 'Дети', and 'ФИО'. The 'ФИО' column displays the concatenated values of the first four columns. The 'Дети' column contains checkboxes, with some rows having them checked. The application window includes a toolbar with navigation and search icons, and a status bar at the bottom.

Код_сотрудника	Фамилия	Имя	Отчество	Должность	Пол	Семейное_положение	Дети	ФИО
1	Петрова	Ирина	Петровна	Официант	женский	замужем	☒	1 Петрова Ирина Петровна
2	Сидоров	Петр	Сергеевич	Официант	мужской	холост	☐	2 Сидоров Петр Сергеевич
3	Иванов	Сергей	Сергеевич	Официант	мужской	женат	☒	3 Иванов Сергей Сергеевич
4	Петров	Иван	Васильевич	Менеджер	мужской	женат	☒	4 Петров Иван Васильевич
5	Сидорова	Мария	Ивановна	Официант	женский	не замужем	☐	5 Сидорова Мария Ивановна
6	Иванов	Егор	Николаевич	Официант	мужской		☐	6 Иванов Егор Николаевич
*							☐	

Если через форму продаж, то форма с выделенным текущим сотрудником в продаже и с кнопкой Выбрать:

Список сотрудников									
2	из 6	Выбрать	Поиск	Фильтр					
	Код_сотрудника	Фамилия	Имя	Отчество	Должность	Пол	Семейное_положение	Дети	ФИО
▶	1	Петрова	Ирина	Петровна	Официант	женский	замужем	☒	1 Петрова Ирина Петровна
	2	Сидоров	Петр	Сергеевич	Официант	мужской	холост	☐	2 Сидоров Петр Сергеевич
	3	Иванов	Сергей	Сергеевич	Официант	мужской	женат	☒	3 Иванов Сергей Сергеевич
	4	Петров	Иван	Васильевич	Менеджер	мужской	женат	☒	4 Петров Иван Васильевич
	5	Сидорова	Мария	Ивановна	Официант	женский	не замужем	☐	5 Сидорова Мария Ивановна
	6	Иванов	Егор	Николаевич	Официант			☐	6 Иванов Егор Николаевич
*								☐	

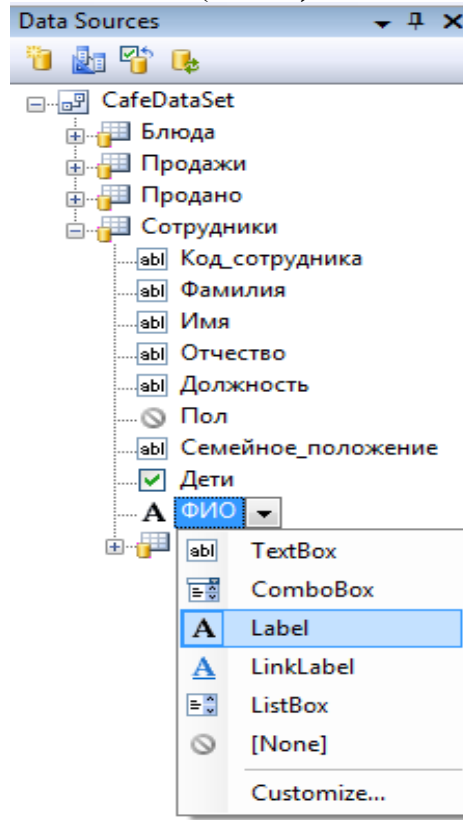
По аналогии создаются и другие вычисляемые поля.

2.6 Создание подстановочных полей (lookup)

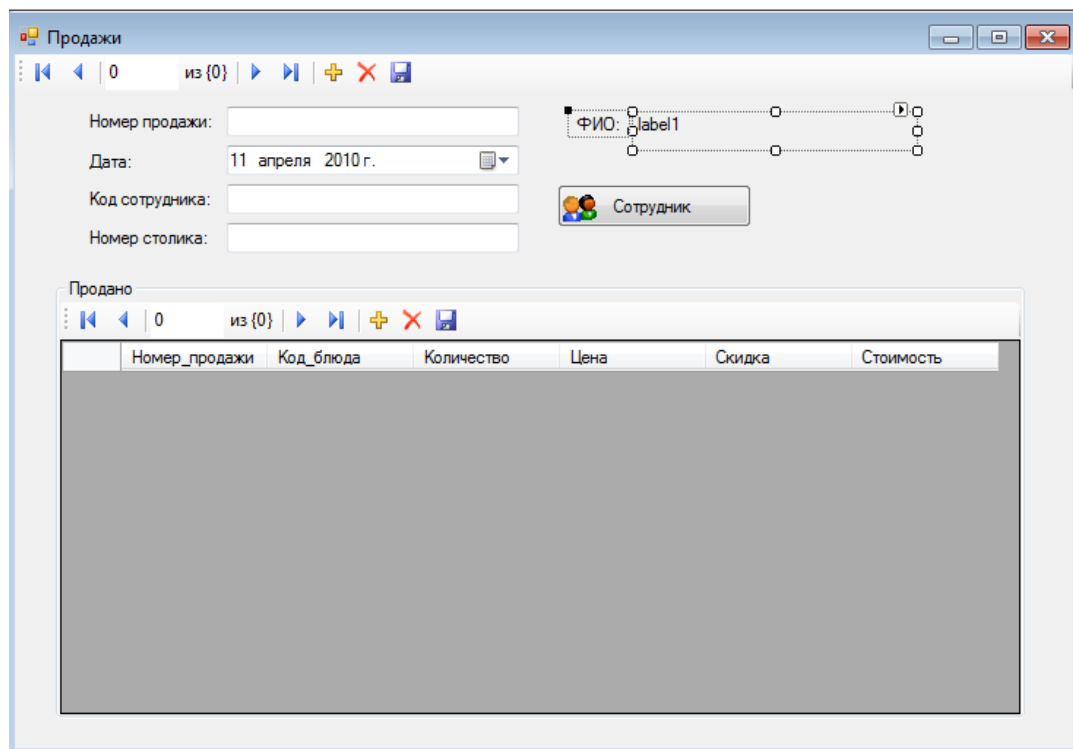
Теперь у нас есть вычисляемое поле ФИО. Можно для удобства пользователей сделать на форме Продажи показ не только кода сотрудника, но и его ФИО с кодом.

Перейдите на форму Продажи (**FormSalesList**).

Выберете из источника данных DataSources на форму данное поле из таблицы **Сотрудники** в виде метки (**Label**):

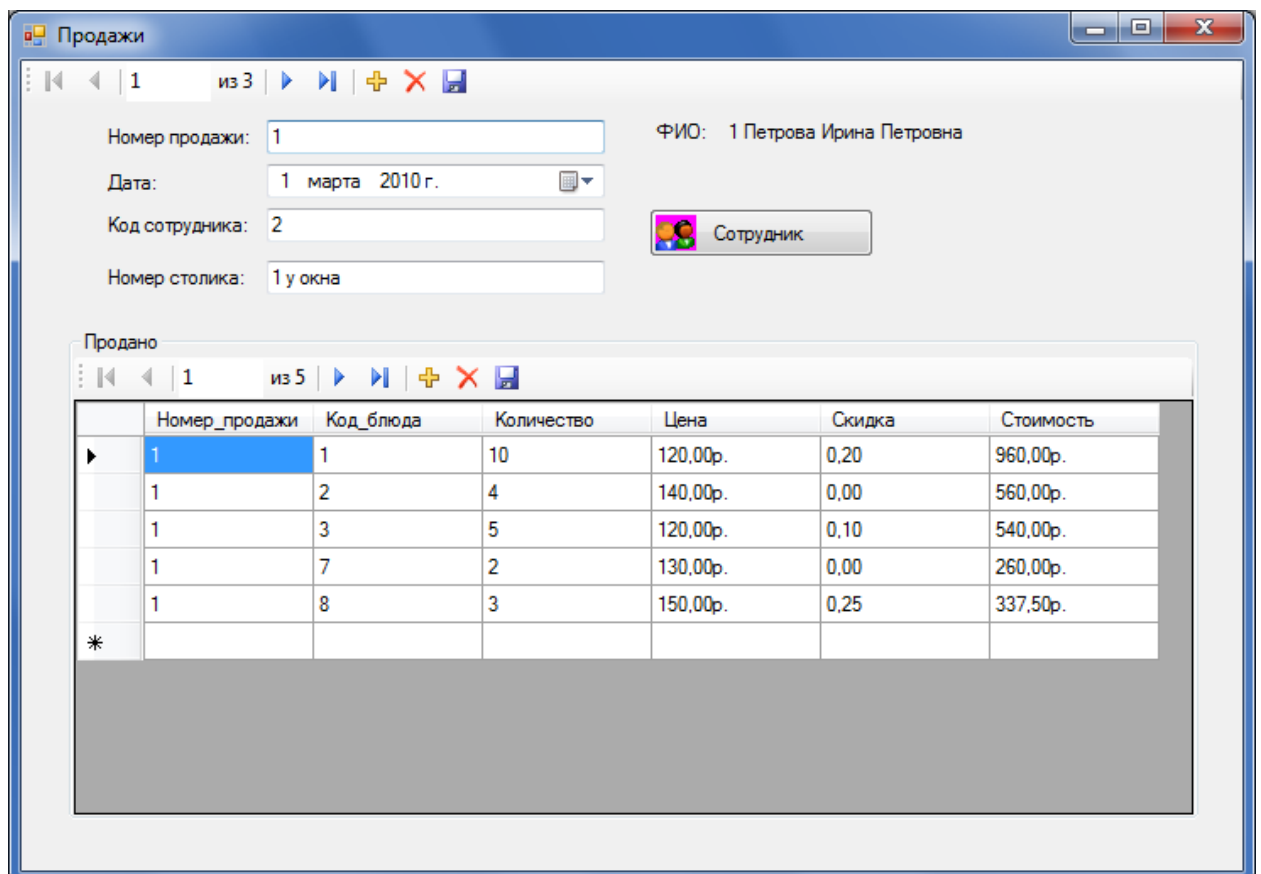


И переместите на форму Продажи (**FormSalesList**):



Запустите программу.

Как Вы видите, значения данных сотрудника в текущей продаже не соответствует тому значению, что показывает ФИО из таблицы **Сотрудники**.



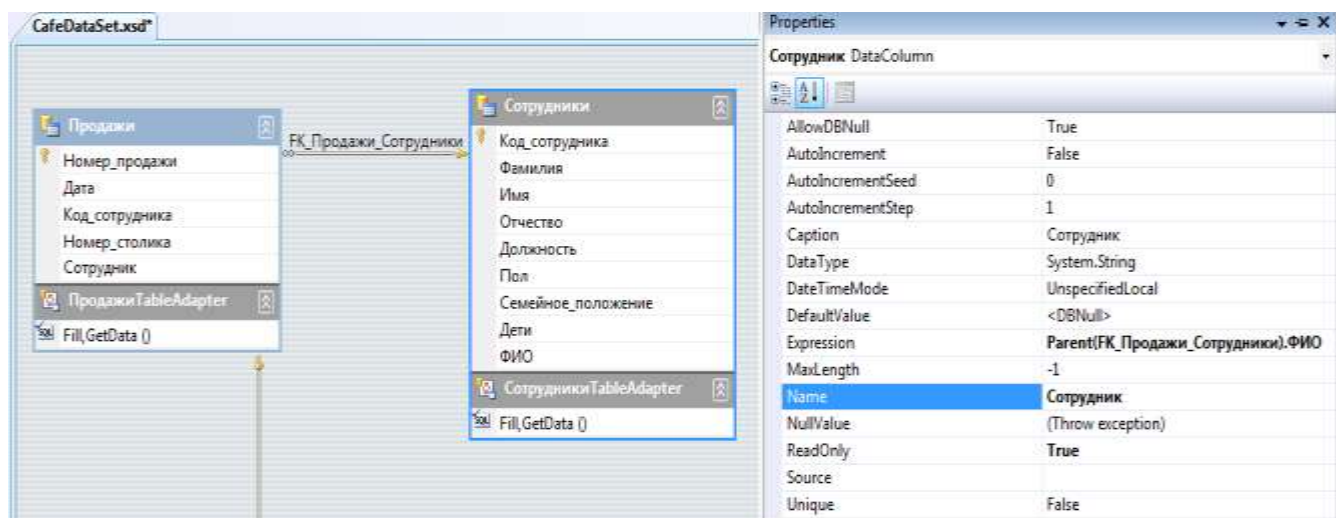
Данную ситуацию можно решить несколькими способами.

Например, проводить поиск для сотрудника в таблице **Продажи** и сотрудника в таблице **Сотрудники** (см. раздел 2.3). Можно создать подстановочные поля (lookup) и др.

Рассмотрим, как создавать подстановочные поля (**lookup**) на примере.

Создадим в таблице **Продажи** новое поле **Сотрудник**, которое будет брать свои значения из таблицы **Сотрудники** из поля ФИО.

Для этого поля **Сотрудник** в свойстве **Expression** укажите:
Parent(FK_Продажи_Сотрудники).ФИО



Где **Parent(родитель)** указывает принадлежность поля ФИО к родительской таблице **Сотрудники** через связь **FK_Сотрудники_Продажи**.

Затем перейдите на форму **FormSalesList** и положите для уточнения под полем Код сотрудника метку с его ФИО:

Продажи

1 из 3

Номер продажи: 1

ФИО: 1 Петрова Ирина Петровна

Дата: 1 марта 2010 г.

Код сотрудника: 2

Сотрудник: 2 Сидоров Петр Сергеевич

Номер столика: 1 у окна

Продано

1 из 5

	Номер_продажи	Код_блюда	Количество	Цена	Скидка	Стоимость
▶	1	1	10	120,00р.	0,20	960,00р.
	1	2	4	140,00р.	0,00	560,00р.
	1	3	5	120,00р.	0,10	540,00р.
	1	7	2	130,00р.	0,00	260,00р.
	1	8	3	150,00р.	0,25	337,50р.
*						

А ФИО с меткой можно удалить с формы:

Продажи

1 из 3

Номер продажи: 1

Дата: 1 марта 2010 г.

Код сотрудника: 2

Сотрудник: 2 Сидоров Петр Сергеевич

Номер столика: 1 у окна

Продано

1 из 5

	Номер_продажи	Код_блюда	Количество	Цена	Скидка	Стоимость
▶	1	1	10	120,00р.	0,20	960,00р.
	1	2	4	140,00р.	0,00	560,00р.
	1	3	5	120,00р.	0,10	540,00р.
	1	7	2	130,00р.	0,00	260,00р.
	1	8	3	150,00р.	0,25	337,50р.
*						

Примечание:

Когда пользователь работает в форме Продажи и открывает из нее форму Сотрудники, то он всегда может не только выбрать уже имеющегося сотрудника, но и добавить нового сотрудника, отредактировать данные по существующим сотрудникам, при желании удалить.

Чтобы подстановочное поле могло брать обновленные записи из таблицы сотрудников **Сотрудники**, необходимо дописать в обработчике **buttonWorker_Click** выделенную строку:

```
private void buttonWorker_Click(object sender, EventArgs e)
{
    int id = -1;
    if
    ((DataRowView)продажиBindingSource.Current) ["код_сотрудника"].ToString() !=
    "")
    {
        id =
        (int) ((DataRowView)продажиBindingSource.Current) ["код_сотрудника"];
    }
    id = FormWorkesList.fw.ShowSelectForm(id);
    if (id >= 0)
    {
        MessageBox.Show(id.ToString());
        ((DataRowView)продажиBindingSource.Current) ["код_сотрудника"]
        = id;
        продажиBindingSource.EndEdit();
        сотрудникиTableAdapter.Fill(this.cafeDataSet.Сотрудники);
    }
}
```

По аналогии создаются и другие подстановочные поля (lookup).

2.7 Связь М:М между Блюда и Продажи в приложении

Связь М:М между **Блюда** и **Продажи** в реляционных БД реализуется через связи 1:М между **Продажи** и **Продано** и 1:М между **Блюда** и **Продано**.

Реализация 1:М между **Продажи** и **Продано** выполнена в разделе 2.1.

Добавим реализацию 1:М между **Блюда** и **Продано**.

Перейдите на форму **FormSalesList**.

Обратите внимание, что при оформлении блюд в продажах необходимо помнить их код блюда, что не всегда является удобным.

Настроим более удобный способ указания блюд в продажах для пользователя.

Наиболее быстрый способ - это настройка подстановочного поля в **Продано**, которое по заданному коду блюда будет брать значения из таблицы **Блюда**, например, его название и тип.

Сформируем такую колонку.

Для этого необходимо создать новую колонку **Инфо** строкового типа в таблице **Блюда**. Для нее укажите в свойстве **Expression**:

Код_блюда+' '+Название+'-'+Тип

The screenshot shows the Visual Studio IDE with the 'Блюда' table selected in the Solution Explorer. The Properties window displays the 'Инфо' DataColumn. The 'Expression' property is highlighted and set to 'Код_блюда+' '+Название+'-'+Тип'.

Property	Value
AllowDBNull	True
AutoIncrement	False
AutoIncrementSeed	0
AutoIncrementStep	1
Caption	Инфо
DataType	System.String
DateTimeMode	UnspecifiedLocal
DefaultValue	<DBNull>
Expression	Код_блюда+' '+Название+'-'+Тип
MaxLength	-1
Name	Инфо
NullValue	(Throw exception)
ReadOnly	True
Source	
Unique	False

Создайте в **Продано** новую колонку **О_блюде** строкового типа. Для него укажите в свойстве **Expression**:

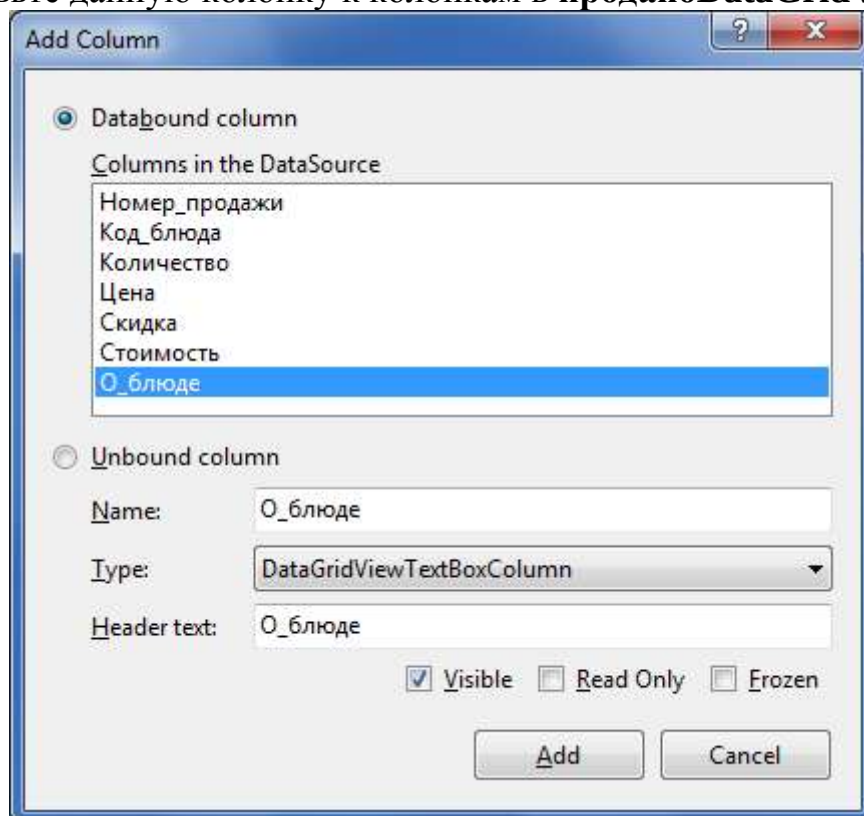
Parent(FK_Продано_Блюда).Инфо

The screenshot shows the Visual Studio IDE with the 'Продано' table selected in the Solution Explorer. The Properties window displays the 'О_блюде' DataColumn. The 'Expression' property is highlighted and set to 'Parent(FK_Продано_Блюда).Инфо'. A diagram on the left shows the relationship between the 'Блюда' and 'Продано' tables, with a line labeled 'FK_Продано_Блюда' connecting them.

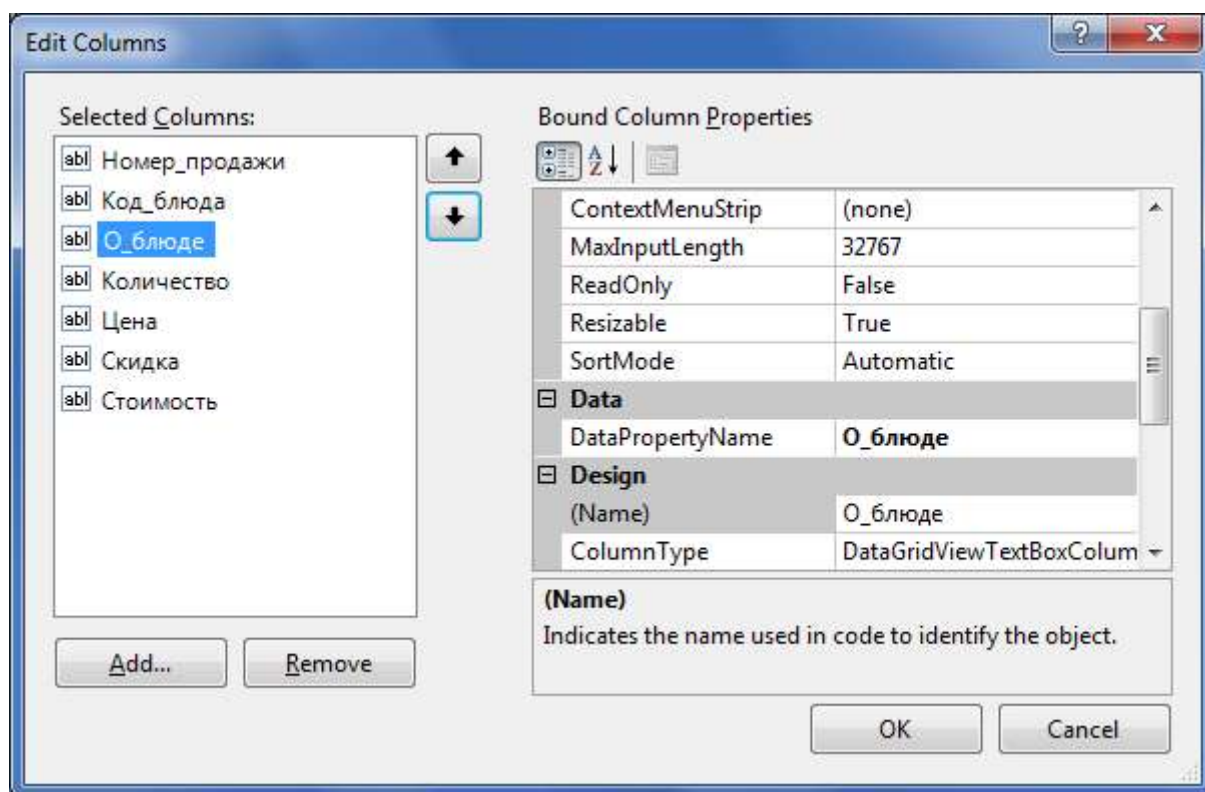
Property	Value
AllowDBNull	True
AutoIncrement	False
AutoIncrementSeed	0
AutoIncrementStep	1
Caption	О_блюде
DataType	System.String
DateTimeMode	UnspecifiedLocal
DefaultValue	<DBNull>
Expression	Parent(FK_Продано_Блюда).Инфо
MaxLength	-1
Name	О_блюде
NullValue	(Throw exception)
ReadOnly	True
Source	
Unique	False

В данном случае для родителя (**Parent**) необходимо указать имя связи **FK_Продано_Блюда**.

Добавьте данную колонку к колонкам в **проданоDataGridView**:



Переместите в колонках **проданоDataGridView** колонку **O_блюда** рядом с **Код_блюда**, выбирая соответствующие стрелки:



Запустите программу. Посмотрите добавленную возможность:

Продажи

1 из 3

Номер продажи: 1

Дата: 1 марта 2010 г.

Код сотрудника: 2

Сотрудник: 2 Сидоров Петр Сергеевич

Номер столика: 1 у окна

Продано

1 из 5

	Номер_продажи	Код_блюда	О_блюда	Количество	Цена	Скидка	Стоимость
▶	1	1		10	120,00р.	0,20	960,00р.
	1	2		4	140,00р.	0,00	560,00р.
	1	3		5	120,00р.	0,10	540,00р.
	1	7		2	130,00р.	0,00	260,00р.
	1	8		3	150,00р.	0,25	337,50р.
*							

Как видите, значения подстановочного поля **О_блюда** не отображаются, так как набор данных для блюд на данной форме не сформирован.

Для его формирования можно, например, добавить поле **Инфо** из **Блюда** на форму. При этом будут размещены:

блюдаBindingSource блюдаTableAdapter

Теперь запустите приложение.

Продажи

1 из 3

Номер продажи: 1

Дата: 1 марта 2010 г.

Код сотрудника: 2

Сотрудник: 2 Сидоров Петр Сергеевич

Номер столика: 1 у окна

Инфо: 1 Грибной-Супы

Сотрудник

Продано

1 из 5

	Номер_продажи	Код_блюда	О_блюде	Количество	Цена	Скидка	Стоимость
▶	1	1	1 Грибной-Супы	10	120,00р.	0,20	960,00р.
	1	2	2 Салат вкусный-Салаты	4	140,00р.	0,00	560,00р.
	1	3	3 Борщ-Супы	5	120,00р.	0,10	540,00р.
	1	7	7 Котлеты-Блюда из мяса	2	130,00р.	0,00	260,00р.
	1	8	8 Торт с вишней-Десерты	3	150,00р.	0,25	337,50р.
*							

Ненужное на данной форме поле **Инфо** можно удалить:

Продажи

1 из 3

Номер продажи: 1

Дата: 1 марта 2010 г.

Код сотрудника: 2

Сотрудник: 2 Сидоров Петр Сергеевич

Номер столика: 1 у окна

Сотрудник

Продано

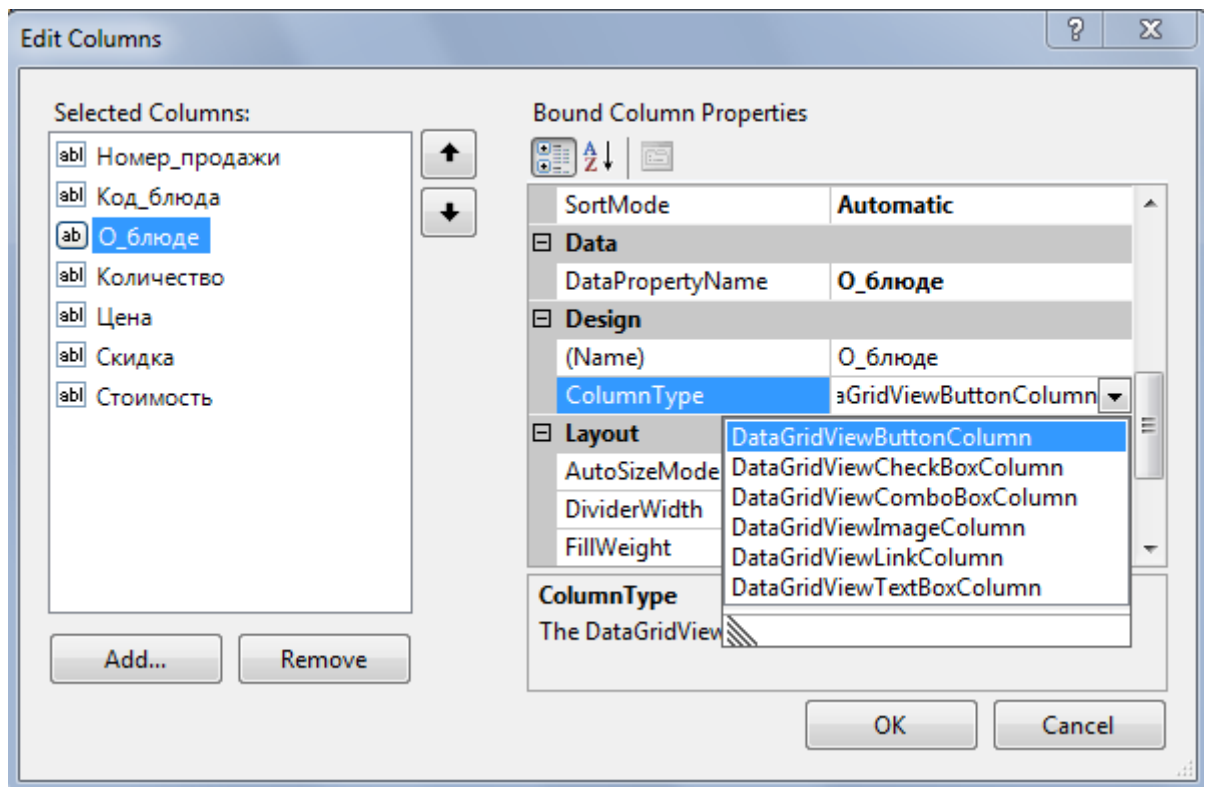
1 из 5

	Номер_продажи	Код_блюда	О_блюде	Количество	Цена	Скидка	Стоимость
▶	1	1	1 Грибной-Супы	10	120,00р.	0,20	960,00р.
	1	2	2 Салат вкусный-Салаты	4	140,00р.	0,00	560,00р.
	1	3	3 Борщ-Супы	5	120,00р.	0,10	540,00р.
	1	7	7 Котлеты-Блюда из мяса	2	130,00р.	0,00	260,00р.
	1	8	8 Торт с вишней-Десерты	3	150,00р.	0,25	337,50р.
*							

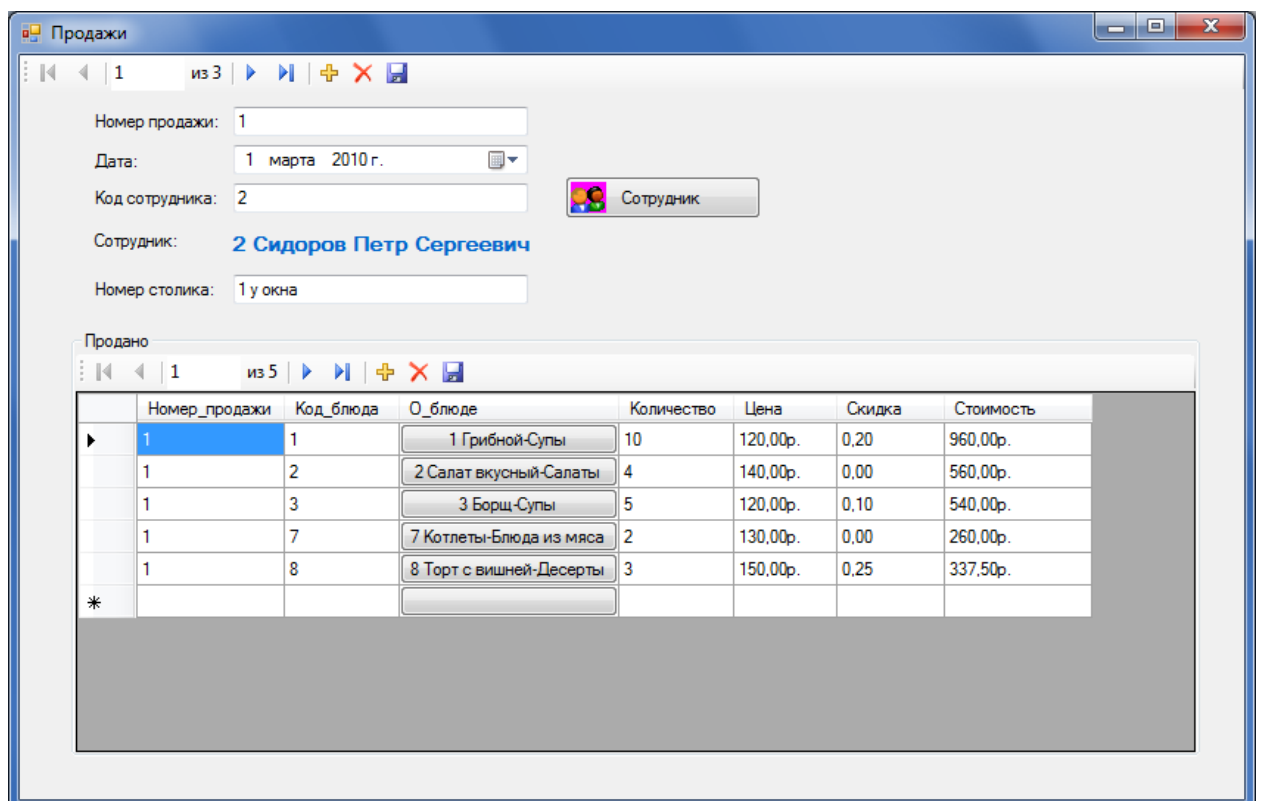
Продолжим оформлять форму с продажами.

Будем показывать меню блюд по нажатию кнопки в проданоDataGridView. Для этого воспользуемся уже добавленной колонкой **О_блюде** (или создайте отдельную колонку).

Укажите для данной колонки тип кнопки (DataGridViewButtonColumn):



Запустите программу. Посмотрите, какие изменения произошли в проданоDataGridView:



Закройте программу. Вернитесь в проект.

Запрограммируем данную кнопку на показ формы с меню блюд по аналогии, как было реализовано для **Сотрудники** и **Продажи** (см. раздел 2.4).

Для удобства заполнения продаж, будем для выбранного блюда указывать его цену из меню блюд при оформлении продажи.

Для этого укажите в коде формы для блюд (**FormDishesList**):

```
int idCurrent = -1;

public int ShowSelectForm(int idDish, out decimal price)
{
    idCurrent = idDish;
    ShowDialog();
    price =
(decimal) ((DataRowView)блюдаBindingSource.Current) ["цена_меню"];
    return
(int) ((DataRowView)блюдаBindingSource.Current) ["код_блюда"];
}
```

В коде **FormDishesList** пропишите:

```
public void ShowForm()
{
    блюдаBindingSource.Position = 0;
    Show();
    Activate();
}

private void FormDishesList_Shown(object sender, EventArgs e)
{
    блюдаBindingSource.Position =
блюдаBindingSource.Find("код_блюда", idCurrent);
}
```

В коде **FormSalesList** пропишите:

```
string GetSelectedFieldName()
{
    return
проданоDataGridView.Columns[проданоDataGridView.CurrentCell.ColumnIndex].Data
PropertyName;
}

private void проданоDataGridView_CellClick(object sender,
DataGridViewCellEventArgs e)
{
    if (String.Compare(GetSelectedFieldName(), "О_блюде", false) ==
0)
    {
        decimal price = 0;
        int idDishCurrent = -1;
```



```

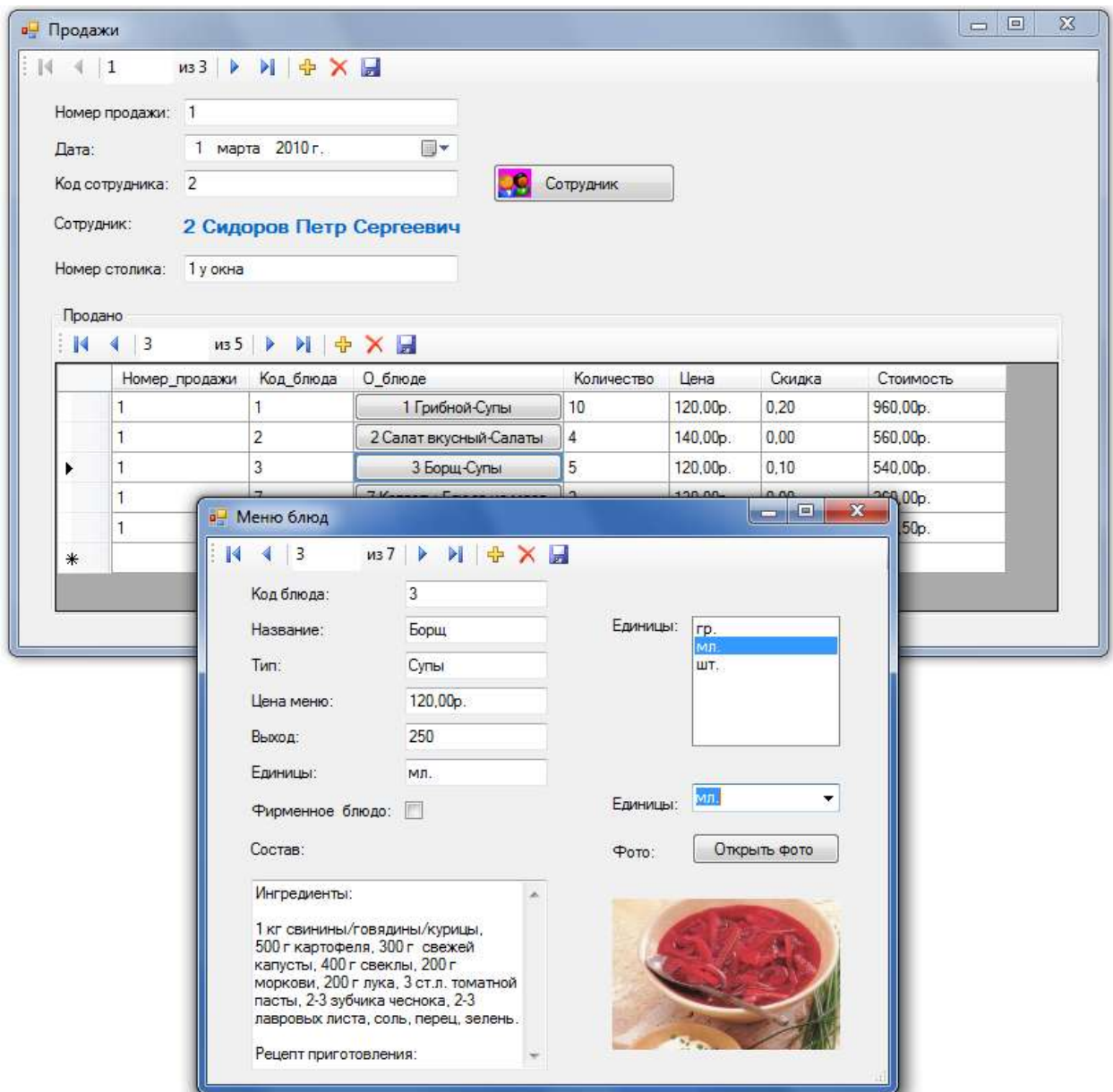
int.TryParse(((DataRowView) проданоBindingSource.Current) ["код_блюда"]).ToString(), out idDishCurrent);
        int idDish = FormDishesList.fd.ShowSelectForm(idDishCurrent,
out price);
        MessageBox.Show("Код блюда=" + idDish.ToString());
        ((DataRowView) проданоBindingSource.Current) ["код_блюда"] =
idDish;

        ((DataRowView) проданоBindingSource.Current) ["цена"] = price;
        проданоBindingSource.EndEdit();
        проданоTableAdapter.Update(this.cafeDataSet);
        блюдаTableAdapter.Fill(this.cafeDataSet.Блюда);
    }
}

```

Запустите программу.

Посмотрите полученную возможность:



Примечание:

Кнопка **Выбрать** в данном примере для формы блюд не была использована. Выбор блюда происходит по закрытию формы.

3 Задание на лабораторную работу №3

3.1 Изучить материал лабораторной работы №3.

3.2 Для своей БД необходимо выполнить:

3.2.1 создать форму/формы проекта для работы с данными по связанным таблицам. Обязательно 1:M на одной форме, M:M через 1:M и M:1 на одной форме;

3.2.2 реализовать шаблон проектирования «Одиночка» (англ. Singleton);

3.2.3 обработать исключения в программе, возникающие при некорректной работе с данными (не менее 2-3 разных ошибок);

3.2.4 реализовать поиск, фильтрацию данных;

3.2.5 создать вычисляемые колонки (не менее 1-2 разных);

3.2.6 создать подстановочные колонки (не менее 1-2 разных);

3.2.7 использовать различные элементы управления Label, TextBox, Button, DataGridView, BindingNavigator и др.;

3.2.8 настроить для удобства BindingNavigator (всплывающие подсказки обязательно);

3.2.9 протестировать работу полученного приложения.

4 Оформление отчета к лабораторной работе

Оформить отчёт со следующим содержанием:

4.1 Титульный лист.

4.2 Цель работы.

4.3 Постановка задачи.

4.4 Краткая теория и ход выполнения заданий.

4.5 Описание результатов.

Приведите скриншоты полученной программы и листинг программы.

4.6 Заключение (выводы).

Отчет должен быть оформлен согласно требованиям ОС ТУСУР.

5 Контрольные вопросы

- 5.1** Что характерно для ссылочной целостности?
- 5.2** Что подразумевает каскадное удаление и каскадное обновление?
- 5.3** Как можно оформить в приложении работу с данными по связанным таблицам БД?
- 5.4** Какие ошибки могут вызываться компьютерной программой?
- 5.5** Что такое исключение? При помощи какой конструкции можно обрабатывать исключения, возникающие при некорректной работе с данными?
- 5.6** Что характерно для конструкции try...catch?
- 5.7** Как можно реализовать поиск, фильтрацию данных?
- 5.8** Особенности метода BindingSource.Find.
- 5.9** Особенности свойства BindingSource.Filter.
- 5.10** Как можно создать вычисляемые колонки?
- 5.11** Как можно создать подстановочные колонки? Что характерно для подстановочных колонок?
- 5.12** Каковы особенности работы со связанными данными в приложении?